



EÖTVÖS LORÁND TUDOMÁNYEGYETEM

INFORMATIKAI KAR

PROGRAMOZÁSI NYELVEK ÉS FORDÍTÓPROGRAMOK

TANSZÉK

Többjátékos 3D labirintusjáték megvalósítása OpenGL-ben

Témavezető:

Pataki Norbert

egyetemi docens

Szerző:

Széles Kristóf

programtervező informatikus BSc

Budapest, 2021

Tartalomjegyzék

1. Bevezetés	3
2. Felhasználói dokumentáció	5
2.1. Rendszerkövetelmények	5
2.2. A játékprogram (maze-game)	7
2.2.1. Konfiguráció	8
2.2.2. Vezérlés	9
2.2.3. Egyjátékos mód (Single Player)	10
2.2.4. Többjátékos mód (Multiplayer)	13
2.2.5. Pályaszerkesztő (Map Editor)	15
2.3. A szerverprogram (maze-server)	17
2.3.1. Konfiguráció	18
2.3.2. Parancsok	19
3. Fejlesztői dokumentáció	20
3.1. A fordításhoz szükséges eszközök	20
3.1.1. Fordítás Windows rendszeren	20
3.1.2. Fordítás Linux rendszeren	20
3.2. A játékprogram (maze-game)	21
3.2.1. A Game osztály	22
3.2.2. A Config osztály	27
3.2.3. Az Entity osztály	27
3.2.4. A Position struktúra	30
3.2.5. A Map osztály	30
3.2.6. A Camera osztály	31
3.2.7. A Client osztály	32

3.2.8. Az Editor osztály	35
3.2.9. A Mesh osztály	37
3.2.10. A Dimension struktúra	41
3.2.11. A Maze osztály	41
3.2.12. A Texture osztály	45
3.2.13. A Window osztály	46
3.2.14. A Button osztály	47
3.2.15. A Menu osztály	48
3.2.16. Főprogram (main.cpp)	48
3.2.17. Segédfüggvények (drawutils.h)	49
3.3. A szerverprogram (maze-server)	51
3.3.1. A Server osztály	51
3.3.2. A ServerThread osztály	53
3.3.3. A Map osztály	55
3.3.4. Az Entity osztály	55
3.3.5. A Config osztály	55
3.4. Algoritmusok	56
3.4.1. A randomizeDfs algoritmus	56
3.4.2. A backtrackIter algoritmus	57
3.5. Tesztelés	58
 4. Összegzés	 60
 Irodalomjegyzék	 61
 Ábrajegyzék	 62
 Táblázatjegyzék	 63
 Forráskódjegyzék	 64

1. fejezet

Bevezetés

Szakdolgozatom célja egy háromdimenziós, belső nézetes labirintusjáték megvalósítása C++ programozási nyelven. A megjelenítéshez OpenGL-t fogok használni segítségül. Az OpenGL egy olyan többnyelvű, multiplatformos alkalmazásprogramozási felület (API), amellyel közvetlenül, parancsokkal vezérelhetjük a grafikus processzort (azaz a GPU-t), ezáltal megvalósítható a 2D-s, vagy 3D-s alakzatok képernyőre rajzolása. Egy OpenGL kontextust többféleképpen is lehet implementálni, különböző programkönyvtárak segítségével. Ilyenek például a FreeGLUT, GLFW, SDL, vagy SFML.

Mivel a szakdolgozat egy játék megírására specializálódott, ezért ehhez az SDL (azaz Simple DirectMedia Layer) könyvtárat [1] fogom alkalmazni. Az SDL lényegében egy C nyelven alapuló multimédiás könyvtár, amellyel nem csak egy OpenGL kontextust lehet létrehozni, hanem lehetséges vele a bemeneti eszközök kezelése, hangok lejátszása, vagy a hálózatkezelés, a többjátékos (multiplayer) játékok elkészítéséhez.

A játékmotor az objektumorientált programozás (OOP) paradigmáját követi, így a program kódja az objektumok révén könnyen átlátható, valamint jól karbantartható és bővíthető. A főbb komponensek (például pályák, entitások, poligonhálók) különböző osztályokba szerveződnek. Az osztályokról részletesebben szó fog esni a fejlesztői dokumentációban. Az osztályok absztrakt reprezentációja miatt többféleképpen implementálható az egyes komponensek működése, így akár teljesen egyedi játékokat is tudunk készíteni. Egy labirintusjátékot fogok benne elkészíteni, amelyben a pályák gráfelméleti algoritmussal automatikusan generáltak, illetve egy be-

épített pályaszerkesztővel lehetőség van saját, egyedi pályák készítésére is. Egy 3D labirintusjáték szerintem nagyon jól szemlélteti a játékmotor adta lehetőségeket. A labirintusok tulajdonképpen pályák, amelyek entitásokból (játékosokból, falakból stb.) állnak és az entításokhoz háromdimenziós modellek tartoznak, ezáltal a háromdimenziós térben tudjuk reprezentálni az elhelyezkedésüket és a megjelenésüket.

A többjátékos mód megvalósításával bemutatom a TCP alapú kliens-szerver kommunikációt az `SDL_net` hálózatkezelési könyvtár segítségével. A hálózati kommunikáció a kliensek (játékosok), valamint egy többszálú (multithreaded) szerverprogram között zajlik. Ezen játékmód kialakítása a játékelményhez is nagyban hozzájárul, ugyanis ekkor másokkal is játszhatunk egy játékot valós időben, ugyanazon a pályán.

2. fejezet

Felhasználói dokumentáció

A felhasználói dokumentáció olyan segédletet nyújt a felhasználó számára, amely nem igényel programozói ismereteket és könnyen elsajátíthatóvá válik a szoftver használata. Részletes leírást ad a futtatásról, a funkciók működéséről és a konfigurálhatóságról.

A játékmotor egy labirintusjátékot valósít meg, amelyben egy- vagy több játékos bolyonghat véletlenszerűen generált labirintusokban, vagy a pályaszerkesztővel készített egyedi pályákon. A szoftver két fő egységre bontható: A kliensoldalon futó játékprogramra (**maze-game**), valamint a szerverprogramra (**maze-server**). Ezen fő egységek bővebb leírása a következő alfejezetekben (2.2, 2.3) lesz olvasható.

2.1. Rendszerkövetelmények

A szoftver az implementációnak köszönhetően multiplatformos, azaz több különböző rendszeren is futtatható. Elsősorban Windows és Linux rendszereken teszteltem a működését, de számtalan más rendszerre is könnyen átfordítható a megfelelő eszközök segítségével. A program fordításáról a fejlesztői dokumentációban (3. fejezetben) esik szó.

Minimális hardverkövetelmények:

- Legalább 2 magos 2 GHz-es processzor (Intel vagy AMD)
- Minimum 4 GB belső rendszermemória
- OpenGL 3.0 (vagy későbbi) támogatottsággal rendelkező videokártya

Szoftverkövetelmények:

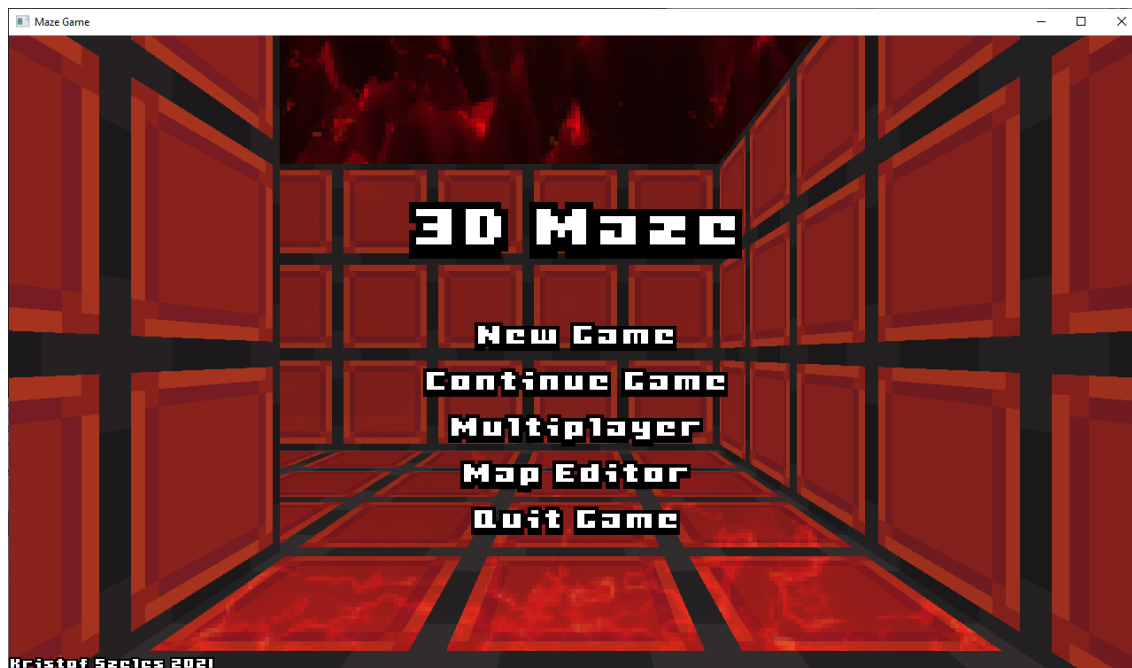
- Windows operációs rendszer esetén Windows 7 vagy újabb kiadás
- Linux operációs rendszer esetén elsősorban Debian alapú rendszerek (például Ubuntu, Linux Mint)

Debian alapú rendszereken program a következő csomagokat igényli a futtatáshoz:

- libSDL2-2.0-0
- libSDL2-image-2.0-0
- libSDL2-net-2.0-0
- libglew2.1

2.2. A játékprogram (maze-game)

A játékprogram a `Release` mappában található `maze-game.exe` futtatásával indítható el Windows rendszeren. Induláskor a felhasználót a játék főmenüje fogadja, ami a 2.1 ábrán látható. A háttérben látható egy demó mód, amelyben a program automatikusan generál egy labirintust és "végigsétál" rajta. Ez a játékelménybe nyújt közvetlen betekintést.



2.1. ábra. A játékprogram főmenüje

A felhasználó a választása szerint a program különböző részeire kerülhet. A menüpontok bővebben az alfejezetekben vannak részletezve. A főmenü a következő menüpontokból áll:

Menüpont neve	Leírás
<i>New Game</i>	Új játék indítása egyjátékos módban
<i>Continue Game</i>	Legutóbbi játék folytatása egyjátékos módban
<i>Multiplayer</i>	Játék indítása többjátékos módban
<i>Map Editor</i>	Beépített pályaszerkesztő indítása
<i>Quit Game</i>	Kilépés a játékból

2.1. táblázat. A játékmenü leírása

2.2.1. Konfiguráció

A játékprogram konfigurációját a `game-config.json` fájl tárolja. Ha a fájl nem létezik, akkor a program automatikusan létrehozza az alapértelmezett beállításokkal. A fájl az indításkor betöltődik, ami meghatározza bizonyos komponensnek működését. Kilépéskor a fájl mentésre kerül, azaz a felhasználó beállításai perzisztens módon "megjegyzésre" kerülnek. A fájl a következő konfigurációs paramétereket definiálja:

- **game:** játékmotor beállításai
 - **cameraFov:** kamera látószögének mérete fokokban
 - **renderDistance:** látótávolság sugara egységekben
 - **mouseSensitivity:** egérérzékenység (0.1 és 1.0 közötti érték, 0.1 - leglassabb, 1.0 - leggyorsabb)
 - **vehicle:** aktuálisan kiválasztott jármű típusának száma
 - **singlePlayer:** egyjátékos mód beállításai
 - * **score:** játékos pontszáma a legutóbbi játékban
 - * **health:** játékos életeréje a legutóbbi játékban
 - * **highScore:** eddigi legnagyobb pontszám
- **maze:** labirintusgenerátor beállításai (**easy**, **medium**, **hard**)
 - **width:** labirintus szélessége
 - **height:** labirintus magassága
- **multiplayer:** többjátékos mód beállításai
 - **defaultHost:** szerver hosztneve (IP címe)
 - **defaultPort:** szerver portszáma
 - **playerName:** játékos neve
- **window:** programablak beállításai
 - **defaultWidth:** az ablak alapértelmezett szélessége
 - **defaultHeight:** az ablak alapértelmezett magassága
 - **maximized:** az ablak maximalizálva van-e

2.2.2. Vezérlés

A játékos irányítása a billentyűzet, valamint az egér használatával történik. A billentyűzeten lévő nyilakkal, illetve a W, A, S, D gombokkal mozgatható a játékos a pályán. Az egér segítségével pedig körül lehet nézni a világban, azaz elforgatható, valamint dönthető a kamera iránya. A gombokhoz rendelt eseményekről részletesebb leírást ad az alábbi táblázat:

Esemény	Leírás
<i>Billentyűlenyomás (F2)</i>	Kameramód váltása FPS (belső nézetes) és TPS (hátnézetes) mód között
<i>Billentyűlenyomás (F11)</i>	A programablak maximalizálása/-visszaállítása
<i>Billentyűlenyomás (W, vagy felfele nyíl)</i>	Játékos mozgatása előre
<i>Billentyűlenyomás (S, vagy lefele nyíl)</i>	Játékos mozgatása hátra
<i>Billentyűlenyomás (A, vagy balra nyíl)</i>	Játékos mozgatása balra
<i>Billentyűlenyomás (D, vagy jobbra nyíl)</i>	Játékos mozgatása jobbra
<i>Billentyűlenyomás (ESCAPE)</i>	Kilépés a főmenübe
<i>Egér mozgítás jobbra/balra, fel/le</i>	Kamera forgatása (yaw - elforgatás az Y tengelyen, pitch - döntés a Z tengelyen)
<i>Egér görgetés fel/le</i>	Kamera zoomolása TPS (hátnézetes) módban előre/hátra
<i>Egérgomb lenyomás (jobb)</i>	Kurzor elfogása/elengedése

2.2. táblázat. A játék vezérlése

2.2.3. Egyjátékos mód (Single Player)

Egyjátékos módban a felhasználó (játékos) egy megadott nehézségű, véletlenszerűen generált labirintusban bolyonghat és különféle kincsek felszedésével pontokat gyűjthet. Azonban a játékos nincs egyedül: A pályán számítógép által irányított járművek (NPC-k) furikáznak, amelyekkel ütközve a játékos életerőt veszít. Ha az életerő eléri a nullát, akkor a játék véget ér (Game Over) és az elért pontszám megjelenik a képernyőn az eddigi legjobb pontszámmal együtt, ezt illusztrálja a 2.2 ábra.

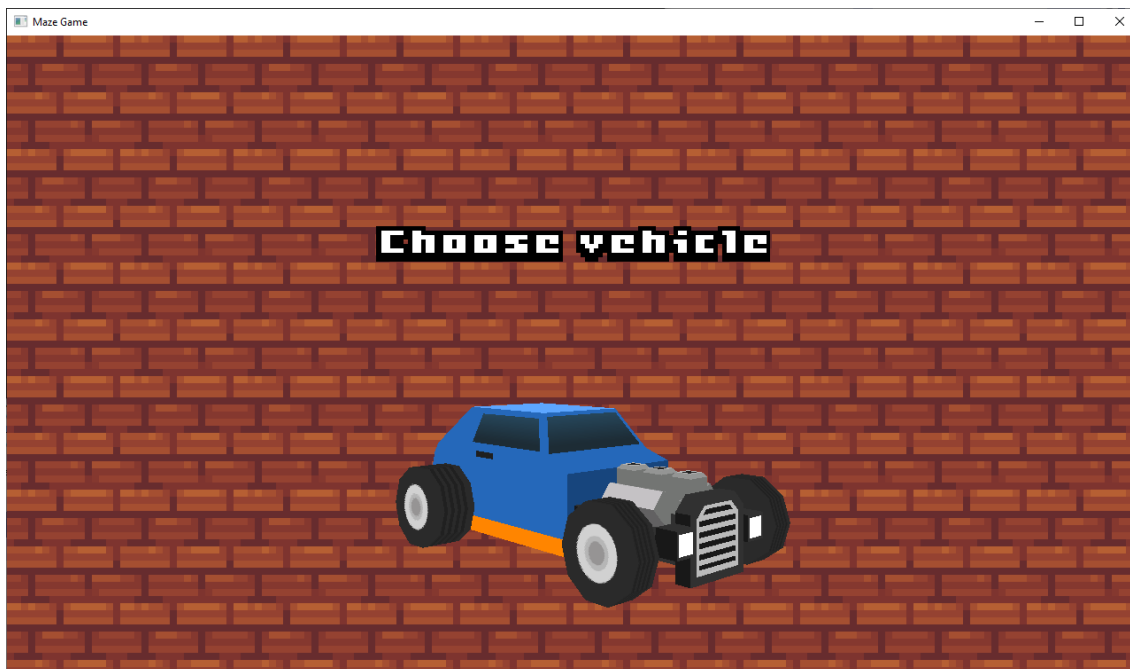


2.2. ábra. Game Over képernyő egyjátékos módban

A főmenüben a **Continue Game** opcióra kattintva lehetőség van a legutóbb játszott játék folytatására, ha volt korábban mentés. Ha még nem volt, akkor új játékot kell kezdeni. A 2.3 ábra alapján látható, hogy új játék esetén kiválasztható a használni kívánt jármű. A billentyűzetten található bal és jobb nyilakkal lehet válogatni a járművek között, majd az **ENTER** gomb lenyomása után a következő lépésben megadható a játék nehézségi szintje, ami alapján a program egy labirintust generál véletlenszerűen. Ezen nehézségi szintek a konfigurációs fájlban vannak definiálva és az alábbiak érhetők el:

- Easy: könnyű

- Medium: közepes
- Hard: nehéz



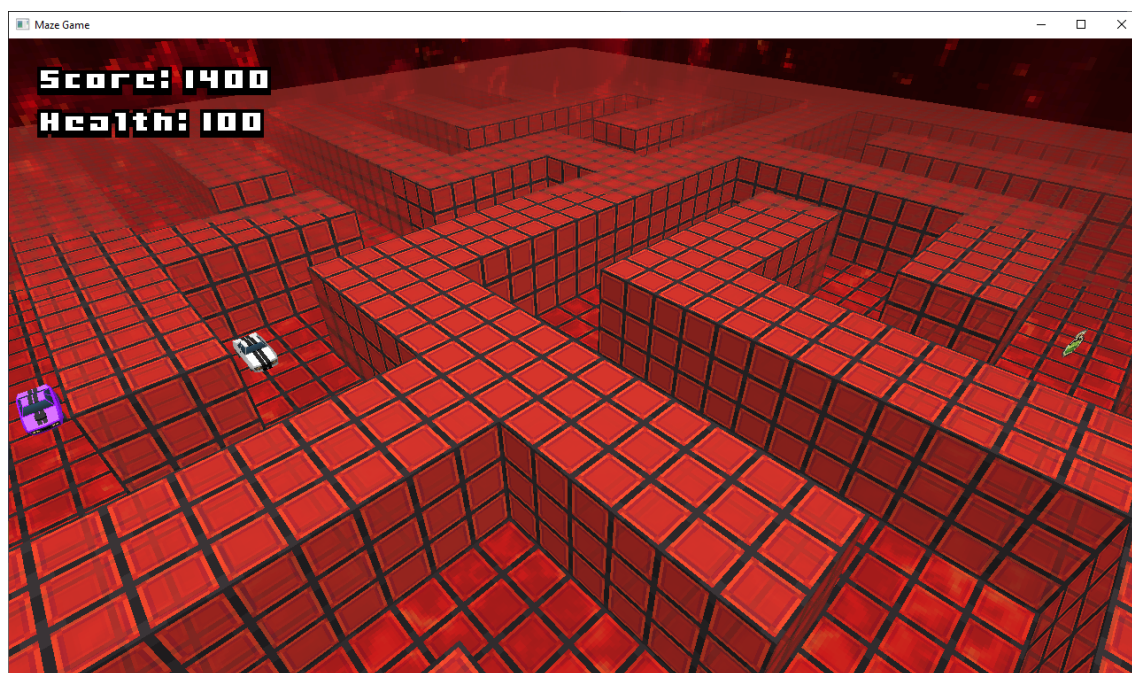
2.3. ábra. Jármű kiválasztása

Továbbá lehetőség van a pályaszerkesztővel készített pályát is betölteni a Custom Map... opcióra kattintva. A labirintus bonyolultsága lehet nagyon nehéz, vagy nagyon egyszerű, ez leginkább a méreteitől függ.

A pályákon különböző kincsek találhatóak, amelyek összegyűjtésével a játék során bónuszpontokat lehet szerezni. Az összegyűjtött pontszám a bal felső sarokban látszódik. Emellett fellelhetők különböző bájitalok is, amelyekkel gyorsítható, vagy éppen lassítható a játékos mozgása. A játékban található tárgyakat és az általuk kiváltott hatásukat az alábbi táblázat mutatja:

Tárgy neve	Leírás
 <i>Gem (ékkő)</i>	A játékos 1000 pontot kap
 <i>Ruby (rubint)</i>	A játékos 800 pontot kap
 <i>Emerald (smaragd)</i>	A játékos 500 pontot kap
 <i>Gold (arany)</i>	A játékos 400 pontot kap
 <i>Fast Potion (gyorsító bájital)</i>	A játékos mozgása 5 másodpercre felgyorsul
 <i>Slow Potion (lassító bájital)</i>	A játékos mozgása 10 másodpercre lelassul

2.3. táblázat. A labirintusban fellelhető tárgyak



2.4. ábra. Közepes nehézségű labirintus egyjátékos módban

Egy bájital felvétele esetén a képernyő jobb felső sarkában megjelenik annak neve, mellette egy számláló, amely visszaszámol, amíg tart a hatása. Ezáltal a játékos tudhatja, hogy egy ilyen ital hatása mennyi ideig fog tartani. Ha egy másik hatása még tart, akkor az utólag felvett bájital hatása kiváltja az előző hatását.

A kijárat megtalálása esetén a játékos 10 ezer pontot kap, majd egy újabb labirintusba kerül, amely más felépítéssel rendelkezik, valamint a tárgyak és az NPC-k is máshol helyezkednek el, a számuk és fajtájuk is véletlenszerű.

2.2.4. Többjátékos mód (Multiplayer)

Többjátékos módban több felhasználó is részt vehet egy labirintusjátékban. Először a játékos nevét kell megadni, ami alapján azonosítható lesz a pályán. Az ENTER gomb lenyomásával lehet a következő képernyőre lépni, ahol kiválasztható egy jármű, amelyet a többi játékos fog látni, ez lesz a játékos "karaktere". Végül a szerver IP címét és a portszámát kell beírni kettősponttal elválasztva, ahova a kliens csatlakozni fog. A CTRL+V billentyűkombinációt alkalmazva beilleszthető a vágólap tartalma, így nem szükséges kézzel begépelni a hosszú címeket.

Lehetőség van más, eltérő játékosokkal és labirintussal rendelkező szerverekre is csatlakozni. Csatlakozás után mindenki ugyanabban a labirintusban vesz részt és a játékosok látják egymás karakterét, tudnak versenyezni, hogy ki fogja előbb megtalálni a kijáratot. A kijárat megtalálása a lehető legtöbb pontot éri. Emellett a pályán szétszórt kincsek felszedésével is tudnak bónusz pontokat gyűjteni.



2.5. ábra. Többjátékos módban résztvevő játékosok

Toplista

A 2.5 ábrán is látható, hogy a bal felső sarokban található a toplista, amely a szerver által számontartott játékosok pontszámait tartalmazza sorszámokkal ellátva. Legfelül, az első helyen az eddigi legtöbb pontot elért játékos található, utána

a többi, csökkenő sorrendben. A pontszámok listája automatikusan frissül, amint valaki pontot szerez, belép, vagy kilép a játékból. Ha egy játékos kilép a szerverről, akkor a pontszáma törlődik és a következő belépésnél újra kell kezdenie a játékot.

Chat ablak

A képernyő bal alsó sarkában található egy chat ablak, ahol a legutóbbi 5 másodpercben fogadott 5 üzenet látható. Itt jelennek meg a szerver által küldött üzenetek (például ha egy játékos csatlakozott, vagy megnyerte a játékot). A játékosok küldhetnek egymásnak saját, rövid chat üzeneteket is, így a játék közben tudnak írásban beszélgetni. Ezt a T (talk) gomb megnyomásával lehet kezdeményezni. Ilyenkor megjelenik egy szövegmező, ahova beírható a küldeni kívánt üzenet, majd az ENTER gomb lenyomásával elküldhető. Az ESCAPE gombbal pedig bezárható a chat ablak, az üzenet elküldése nélkül.

A szerver által küldött üzenetek a következők lehetnek:

Üzenet	Leírás
<i><játékosnév> has connected to the server</i>	Az adott nevű játékos csatlakozott a szerverhez
<i><játékosnév> has disconnected from the server</i>	Az adott nevű játékos kilépett a szerverről
<i><játékosnév> has picked up an item</i>	Az adott nevű játékos felvett egy tárgyat a pályán
<i><játékosnév> has won the game</i>	Az adott nevű játékos megnyerte a játékot
<i><játékosnév>: <üzenet></i>	Az adott nevű játékos chat üzenetet küldött

2.4. táblázat. A szerver által küldött üzenetek

2.2.5. Pályaszerkesztő (Map Editor)

A beépített pályaszerkesztővel tetszőleges felépítésű pályák készíthetők kétdimenziós nézetben. Működése leginkább egy rajzprogramhoz hasonló, viszont itt képpontok helyett blokkok (falak, tárgyak stb.) rajzolhatók a képernyőre. A használatához nem szükségesek különösebb ismeretek, csupán elegendő az egér használata.

A képernyő alján található menüsoron lévő gombokkal vezérelhető a program működése, illetve ezen funkciók a billentyűzet segítségével is elérhetők. A kiválasztott típusú blokk a kurzor alatt megjelenik előnézetként és az egér bal gombját lenyomva elhelyezhető a pályán, a jobb gombbal pedig törölhetők az elhelyezett blokkok. A kiválasztott típust görgetéssel (fel/le) lehet változtatni. A megjelenő blokkok közül az "N" betű az NPC-et jelöli. Nagyobb pályák építése esetén a kamera mozgatható az egér mozgatásával a középső gombot nyomva tartva.



2.6. ábra. Pályaszerkesztő 100×100-as generált labirintussal

A gyorsbillentyűk és a program által kezelt események leírását a következő táblázat tartalmazza:

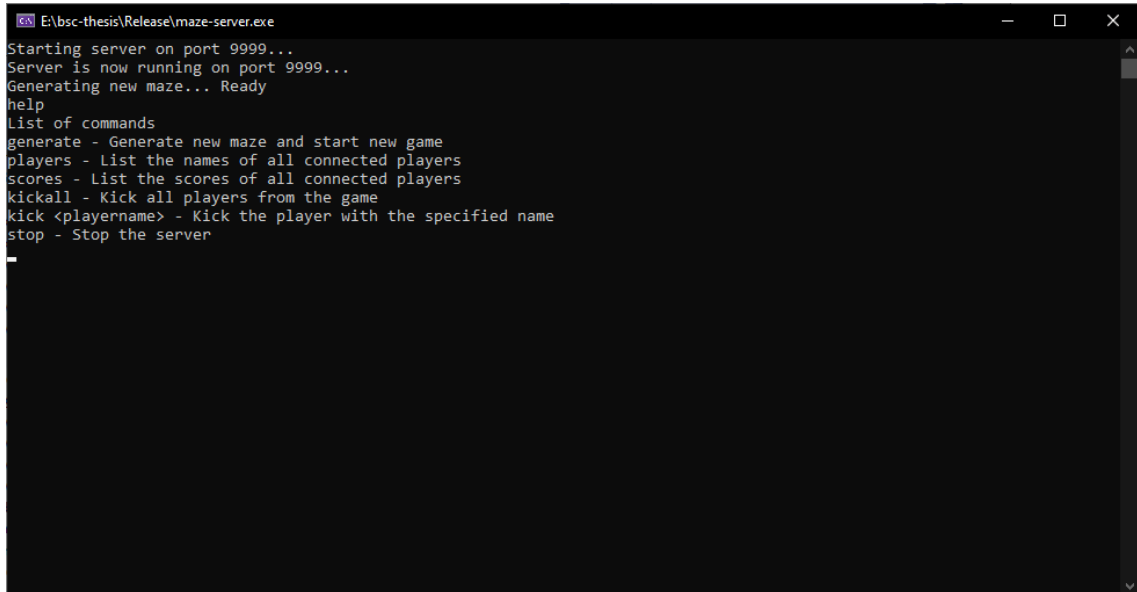
Esemény	Leírás
<i>Billentyűlenyomás (F1)</i>	Új, üres pálya létrehozása
<i>Billentyűlenyomás (F2)</i>	Új, véletlenszerű labirintus generálása a megadott paraméterekkel (szélesség, magasság)
<i>Billentyűlenyomás (F3)</i>	Pálya megnyitása és betöltése a szerkesztőbe (dialógusablakban kiválasztható a fájl)
<i>Billentyűlenyomás (F4)</i>	Szerkesztett pálya mentése fájlba (dialógusablakban kiválasztható a fájl)
<i>Billentyűlenyomás (bal/jobbnyel)</i>	A falak textúrájának változtatása
<i>Billentyűlenyomás (ESCAPE)</i>	Kilépés a főmenübe
<i>Egérgomb lenyomás (bal)</i>	A kiválasztott blokk elhelyezése a kurzor alatt a pályán, vagy a kezdőpont forgatása, ha az volt kijelölve
<i>Egérgomb lenyomás (jobb)</i>	A kurzor alatti blokk törlése a pályáról
<i>Egérgomb lenyomás (középső)</i>	A kamera mozgatása az egér mozgatásával
<i>Egér görgetés (fel/le)</i>	A kiválasztott blokk típusának változtatása
<i>Fájl behúzása az ablakba</i>	Az ablakba behúzott pályafájl betöltése

2.5. táblázat. A pályaszerkesztő eseményeinek leírása

Fontos megjegyezni, hogy mentéskor a program ellenőrzi a bejárat, illetve a kijárat létezését. Amennyiben nem léteznek, a program egy hibaüzenettel felhívja a felhasználó figyelmét ezen blokkok elhelyezésére. Miután elhelyezésre került a bejárat és a kijárat, a következő próbálkozással már menthető a pálya.

2.3. A szerverprogram (maze-server)

A szerverprogrammal lehetőség van játékszervert hosztolni. A program futhat a játékprogram mellett, nem feltétlenül kell külön gépen futnia, valamint a játékprogramot (vagyis a klienst) futtató operációs rendszer is eltérhet a szerverprogram rendszerétől. A program használata észszerű: Indítása után elindul a konfigurációban megadott portszámon, majd elkezdi fogadni a csatlakozó játékosokat.



```
E:\bsc-thesis\Release\maze-server.exe
Starting server on port 9999...
Server is now running on port 9999...
Generating new maze... Ready
help
List of commands
generate - Generate new maze and start new game
players - List the names of all connected players
scores - List the scores of all connected players
kickall - Kick all players from the game
kick <playername> - Kick the player with the specified name
stop - Stop the server
```

2.7. ábra. A szerverprogram konzolablaka futás közben

A program visszajelzi a felhasználónak a sikeres indulást (Server is now running on port XXXXX). Ellenkező esetben egy felugró hibüzenetben jelzi a hiba pontos okát, amiért nem tudott elindulni. A szerver IP címe mindig a futtató gép IP címével egyezik meg, lehetőség van az interneten keresztül is csatlakozni a megfelelő tűzfalbeállítások mellett.

Induláskor a szerverprogram véletlenszerűen generál egy labirintust, amelyet a játékhoz való csatlakozáskor elküld a játékosnak. Amint valaki elérte a célt, a program új labirintust generál.

2.3.1. Konfiguráció

A szerverprogram konfigurációja a `server-config.json` fájlban van tárolva. Ha a fájl nem létezik, akkor a program automatikusan létrehozza az alapértelmezett beállításokkal. A könnyebb átláthatóság kedvéért JSON [2] formátumot alkalmazok, amely egy kulcs-érték párokat tartalmazó objektum. A fájl a következő konfigurációs paramétereket tartalmazza:

- **maze:** labirintusgenerátor beállításai
 - **width:** labirintus szélessége
 - **height:** labirintus magassága
- **server:** szerver beállításai
 - **port:** portszám, amelyen a szerver figyelni fog
 - **maxSlots:** maximális slotok (helyek) száma
 - **mapFile:** a használni kívánt egyedi pályafájl elérési útvonala

A `mapFile` kitöltése esetén a szerverprogram megpróbálja betölteni a pályafájlt a megadott elérési útvonalról. Ha sikeresen megtalálta, akkor induláskor új labirintus generálása helyett a fájlban tárolt pályát fogja betölteni.

2.3.2. Parancsok

A szerverprogram ablakába írva különböző parancsokat lehet kiadni, amellyel befolyásolható a működése. Lényegében adminisztrátorként kezelhetők a játékhoz csatlakozott játékosok. Ezen parancsok listája a **help** paranccsal listázható ki. A program a következő parancsokat kezeli:

Parancs	Leírás
<i>generate</i>	Új labirintus generálása és új játék kezdése
<i>players</i>	Játékosok neveinek kilistázása
<i>scores</i>	Játékosok által elért pontszámok kilistázása
<i>kickall</i>	Összes játékos kirakása a játékból
<i>kick</i> <játékosnév>	Megadott nevű játékos kirakása a játékból
<i>stop</i>	Szerverprogram leállítása és kilépés

2.6. táblázat. A szerverprogram által kezelt parancsok

3. fejezet

Fejlesztői dokumentáció

Ebben a fejezetben a játékmotor működését és felépítését fejtem ki részletesen. Az alfejezetekben, a 3.2 ponttól külön kifejtésre kerülnek a játékot alkotó osztályok használata, illetve a főbb metódusaikhoz biztosított bővebb leírást.

3.1. A fordításhoz szükséges eszközök

A játékmotor a jelenlegi legújabb C++20-as szabványnak megfelelően íródott, ezért a fordításhoz szükség van a legfrissebb fejlesztői eszközök beszerzésére és telepítésére. A következő fejezetekben a különböző platformokra való fordításra adok megoldásokat.

3.1.1. Fordítás Windows rendszeren

Windows rendszereken a fordítás a Visual Studio 2019 fejlesztői környezetben történik. A szoftver ingyenesen elérhető Community verziója számomra a fordításhoz megfelelőnek bizonyult, azonban itt szükség van a C++ támogatás telepítésére. A szakdolgozat mappájában található `sln` fájlt megnyitva betöltésre kerül a két projekt (`maze-game`, `maze-server`), amelyeket azonnal fordítani és futtatni lehet.

3.1.2. Fordítás Linux rendszeren

Linux rendszereken a fordításhoz különböző csomagokat kell telepíteni. A fordítás megkönnyítéséhez projektmappákban (`maze-game`, `maze-server`) lévő shell scripteket

biztosítom segítségül. Ezen fájlok futtatásával kényelmesen végrehajtható állomány generálható a forráskódból a g++ fordítóval.

Debian alapú rendszereken a fordítást megelőzően az alábbi csomagokat kell telepíteni:

- g++-10
- libsdl2-dev
- libsdl2-image-dev
- libsdl2-net-dev
- libgtk-3-dev
- libglm-dev
- libglew-dev
- libglu1-mesa-dev
- zlib1g-dev

3.2. A játékprogram (maze-game)

A játékmotort alapvetően a `Game` osztály reprezentálja. A program indításakor az osztály példányosításra kerül a fő belépési pontként szolgáló `main()` függvényben. Miután létrejött az objektum, meghívásra kerül a `run()` metódusa. Ekkor elindul a játék fő ciklusa, ami egészen addig ismétlődik, amíg a játék ki nem lép, azaz a `quit` adattagja igaz értékű nem lesz. A ciklusmag 3 fő részre bontható:

- rajzolás a képernyőre (`draw` előtagú metódusok)
- felhasználó által kiváltott események kezelése (`handle` előtagú metódusok)
- kirajzolt kép megjelenítése (`updateMapFrame`, `updateMenuFrame`)

3.2.1. A Game osztály

A **Game** osztály a játékmotor fő egységét képezi. Főként a képernyőre rajzoláshoz, illetve az eseménykezeléshez kapcsolódó metódusokat valósítja meg. Az adattagjai között van tárolva például az aktuálisan betöltött pálya (azaz a labirintus), illetve minden olyan változó, konstans, vagy objektum, amelyek a játék működéséhez elengedhetetlenek. Az osztályhoz tartozó főbb objektumok a következők:

- **map**: pálya
- **menu**: játékmnü
- **window**: programablak
- **config**: konfiguráció
- **client**: hálózati kliens
- **drawUtils**: kirajzoláshoz szükséges eszközök

A run metódus

A **Game** osztály **run()** metódusa a játékmotor futtatását végzi. Egy ciklust tartalmaz, amely a kilépésig ismétlődik. A cikluson belül egy elágazás található, amely eldönti, hogy a játékprogram mely részén van a felhasználó és ez alapján milyen műveleteket kell kezelni, illetve végrehajtani. Például, hogy a játékmnüben egy gombra kattintva melyik almenü jelenjen meg. Továbbá itt történik a képernyőre rajzoláshoz tartozó metódusok meghívása (**drawMenu()**, **drawMap()**, **drawHUD()**), amelyek a grafikus felület, illetve a pálya megjelenítését végzik.

A loadMeshes metódus

Az osztály **loadMeshes()** metódusával betöltöm a **models** mappában található **obj** kiterjesztésű fájlokat a **meshes** nevű mapbe. Ez a map tulajdonképpen egy olyan asszociatív tömb, amelyben a modell fájlneve a kulcs, a modellre mutató pointer (**Mesh***) pedig az érték, ezáltal könnyebb rájuk hivatkozni. Az entitásokat a példányosításuk során rendelem hozzá ezekhez a modellekhez. Egy entitáshoz mindig csak egy modell tartozik, de egy modellhez több entitás is tartozhat.

Shaderek

A játékmotor különböző shadereket (vertex shader, fragment shader) használ a háromdimenziós megjelenítéshez. A shader lényegében egy kis program, mellyel a render pipeline különböző szintjeit lehet programozni. Ezen shaderek a `shaders.h` fájlban vannak stringként definiálva, így elérhető, hogy ne kelljen mindig egy külső fájlból betölteni a shader kódját. A program egy vertex shadert és két különböző fragment shadert használ.

A `vertexShader` a játékos és a kirajzolt vertexek távolsága alapján a renderelési távolságot (`renderDistance`) figyelembe véve kiszámítja, hogy egy adott modellnek mekkora legyen az átlátszatlansága (`opacity`). Minél távolabb van egy entitás modellje a játékos pozíciójától (`playerPos`), annál kisebb értéket fog felvenni az átlátszatlanság. A felvett értéke 0,0 és 1,0 között mozoghat, ezért ezt a `clamp()` függvény segítségével lekorlátozom.

A `fragmentShader1` fragment shader akkor kerül felhasználásra, amikor szükség van a háromdimenziós modellek megvilágítására. Ilyenkor adott a fényforrás pozíciója, az erőssége, valamint a színe. Az ambiens és diffúz fények alapján kiszámolom a kirajzolt pixelek színét (`fs_out_col`). Emellett van egy `opacity` bemeneti paramétere is, amellyel a modell átlátszatlansága adható meg. Erre azért van szükség, hogy a renderelési távolsághoz egyre közelebb eső falak átlátszatlansága fokozatosan csökkenjen, ezáltal sokkal látványosabb kép kapható a látótávolság sugaráról.

A `fragmentShader2` használata esetén nincsenek figyelembe véve a fények és mindig olyan színű pixeleket rajzolok ki, mint amilyenek a textúra pixelei, valamint az átlátszatlanság sem szabályozható. Erre például a `skybox` kirajzolásakor van szükség, ugyanis a doboz belső lapjainak azonos fényerejűnek kell lenniük a megfelelő effektus eléréséhez. A `skybox` egy olyan nagy méretű kocka, amelynek belső lapjai valamilyen textúrával rendelkeznek. Ez a kocka körülveszi a pályát, ezáltal olyan hatást kelt, mintha a labirintus valamilyen környezetben lenne (például sivatag), így ennek használata a megfelelő látvány elérése szempontjából elengedhetetlen.

A `drawMap` metódus

A `drawMap()` metódusban a játékmotorba betöltött pályát rajzolom a képernyőre. A játékos pozíciójától függően `skybox`-ot rajzolok a pálya köré a `drawSkybox()`

metódussal, majd kirajzolok a pálya alá egy méretével megegyező padlót a `drawFloor()` metódus segítségével. Végül a `drawEntities()` metódus meghívásával kirajzolom a pálya entitásait reprezentáló poligonhálókat. Ennek működéséről a következő alfejezetben esik szó részletesebben.

A `drawEntities` metódus

A `drawEntities()` metódussal a játékmotorba betöltött pálya (`map`) entitásaihoz tartozó poligonhálókat rajzolom a képernyőre. Először egy `for` ciklussal végigiterálok az entításokat tároló listán (`entities`), majd az `isHidden()` metódussal megvizsgálom, hogy az adott entitás nincs-e elrejtve, valamint, hogy a pozíciója a renderelési távolságon (`renderDistance`) belülre esik-e. Ha a kettő feltétel közül valamelyik nem teljesül, akkor az entitás poligonhálóját nem fog kirajzolásra kerülni. Így elkerülhető, hogy nagy labirintusok kirajzolása esetén a sok rajzolási művelet miatt a játékmotor futása lelassuljon.

A `drawBillboard` metódus

A `drawBillboard()` metódussal a többjátékos módban a játékosok felett megjelenő névtáblák (billboard-ok) kirajzolását végzem. Ezek a billboard-ok olyan egyedi poligonhálókat, amelyek esetén nem alkalmazok forgatást, illetve mélységtesztelést. Ezzel elérhető, hogy a névtáblák mindig a kamera felé "nézzenek" és ne rajzolódnak eléjük más modellek (pl. falak). Így a pályán a falon keresztül látható a többi játékos névtáblája, ezáltal könnyebben meg lehet őket találni.

A metódus működése hasonló a `drawMesh()` működéséhez, azonban itt az előbb említett módon tiltva van a mélységtesztelés (`GL_DEPTH_TEST`), valamint a nézetmátrix (`view`) forgatáshoz tartozó komponense kinullázásra kerül.

A `loadMap` metódus

A `loadMap()` metódusban a paraméterek alapján (`data`, `skyboxTexture`, `tileTexture`) betöltöm a pályát az osztály `map` adattagjába. A `data` paraméter az entítások listája, amely `std::stringstream` típusú, ez alapján fognak betöltődni a pályára a `Map` osztály `loadEntities()` metódusának alkalmazásával.

A metódusnak létezik egy túlterhelt változata, amely csak a betölteni kívánt pályafájl nevét kapja paraméterként. Ilyenkor ebből a fájlból fogja beolvasni az entitások listáját, ami alapján betölti őket a játékmotorba.

A `loadReceivedMap` metódus

A `loadReceivedMap()` metódussal a kliens által fogadott pályaadatok alapján betöltöm a pályát a játékmotorba. Lekérdezem a kienstől a kapott pálya adatait a `getReceivedMap()` metódussal, amelyet paraméterként adok a `loadMap()` metódusnak. Ezután a játékmotorba betöltött pálya memóriacímét a kliensnek adom a `setMap()` metódus alkalmazásával, így az számára közvetlenül elérhetővé válik. Erre szükség van például a játékosok mozgatása esetén, amikor a szervertől fogadott üzenetek alapján mozgatni kell az ellenfeleket (`opponents`) a pályán.

A `handleCollisions` metódus

A `Game` osztály `handleCollisions()` metódusával ütközésvizsgálatot végzek a pályán elhelyezkedő entitások között. Két egymásba ágyazott for ciklussal végigiterálok a pályán elhelyezkedő entitásokon. A külső for ciklusban először veszek egy entitást (`entity1`), majd a belső for ciklusban egy másikat (`entity2`). A másik entitással csak akkor végzek ütközésvizsgálatot, ha a `COLLISION_DISTANCE` konstans által definiált távolságon belül található. Ilyenkor két egymástól messze lévő entitás között nem történik felesleges ütközésvizsgálat és nagy méretű pályák esetén nem okoz lassulást a metódus. Ha a másik entitás az ütközésvizsgálati távolságon kívülre esik, akkor a ciklus a `continue` kulcsszóval továbblép.

Következő lépésként az `entity1 checkCollision()` metódusának paraméterként adva az `entity2`-t eldöntöm, hogy a két entitás ütközik-e. Ha ütköznek (azaz a `checkCollision()` igaz értéket ad vissza) akkor az `entity2` típusától függően különböző műveleteket végzek, például tárgyak felvétele esetén elrejttem a tárgyat, majd pontot adok a játékosnak. Falakkal ütközés esetén a játékos nem mehet át a falon, így szükség van az ütközés feloldására. Ilyenkor meghívom a játékos (`Player`) `resolveCollision()` metódusát, amely megakadályozza, hogy átmenjen a falon, tulajdonképpen "visszanyomja" a falba ütköző játékost.

A `handleMapEvents` metódus

A `handleMapEvents()` metódussal a pályán bekövetkező események kezelését végzem. Egy `while` ciklust tartalmaz, amely addig ismétlődik, amíg az SDL könyvtár `SDL_PollEvent()` függvényhívása igaz értékkel tér vissza, azaz amíg létezik bekövetkezett esemény. Ezután az esemény típusától függően egy `switch` esetszétválasztással különféle eseteket különböztetek meg, például, hogy a felhasználó egérgombot nyomott le (`SDL_MOUSEBUTTONDOWN`), vagy, hogy bezárta a programot (`SDL_QUIT`). Előfordulnak olyan események is, amelyeket a metóduson kívül kezelek le. Ilyenkor visszatérek a bekövetkezett esemény típusával (`MapEvent`).

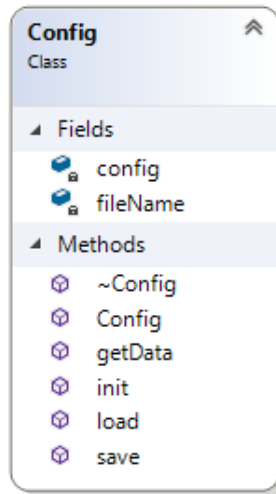
A `handleMapKeyState` metódus

A `handleMapKeyState()` metódus a billentyűzet állapotát figyeli a játékmenet során. Az `SDL_GetKeyboardState()` hívással lekérdezem ezt az állapotot, majd egy esetszétválasztás segítségével megvizsgálom, hogy éppen mely billentyűk voltak lenyomva. Ettől függően végzem a játékos mozgását a pályán, azaz, ha például az előre mutató nyíl gomb volt lenyomva, akkor a játékost a helyzetéhez képest előre felé mozgatom. A játékos mozgása a W, A, S, D gombokkal, valamint a nyílbillentyűkkel történik. Ha egyik ilyen gomb sem volt lenyomva, akkor a játékos gyorsulását 0-ra állítom a `setAcceleration()` metódus használatával. Ilyenkor a játékos mozgása megáll.

A `handleMenuEvents` metódus

A `handleMenuEvents()` metódussal a játékprogram menüjében bekövetkező eseményeket kezelem le, hasonlóan a `handleMapEvents()` metódushoz. A metódus rendelkezik egy egész szám típusú visszatérési értékkel, amely megadja, hogy a menüben felsorolt gombok közül melyik indexűt (`clickedButtonIndex`) nyomta meg a felhasználó. Ezt az eseményt kívül fogom lekezelné a jobb átláthatóság kedvéért. Ha nem történt gombra kattintás, akkor 0-val térek vissza. Ilyenkor vagy egy másik típusú esemény következett be, amely a metóduson belül lekezelésre került, vagy egyáltalán nem következett be esemény.

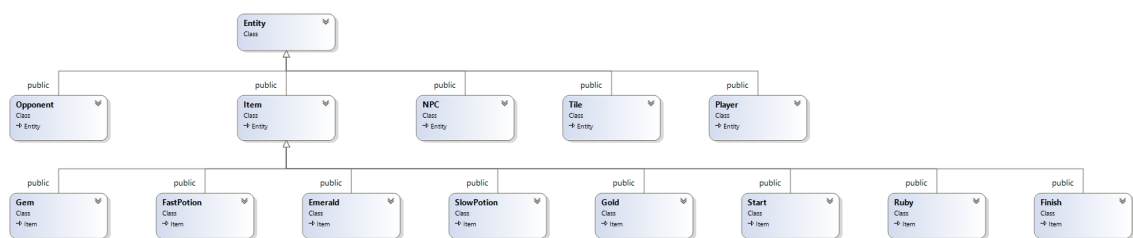
3.2.2. A Config osztály



3.1. ábra. A Config osztály osztálydiagramja

A **Config** osztály a játékmotor konfigurációját tárolja. Konstruktora betölti a paraméterként megadott konfigurációs fájlt a `load()` metódus segítségével. Ha a fájl nem létezik, vagy hibás, akkor automatikusan létrehozom az alapértelmezett beállításokkal. A destruktorban pedig a `save()` metódus meghívásával elmentem az általa tárolt beállításokat a konfigurációs fájlba. Ezáltal nem szükséges manuálisan meghívni a mentéshez és a betöltéshez tartozó metódusokat.

3.2.3. Az Entity osztály



3.2. ábra. Az Entity ősosztály és alosztályainak osztálydiagramja

Az **Entity** absztrakt osztály a játékban létező entitások ősosztálya, ezt illusztrálja a 3.2. ábra. A hasonló tulajdonságokkal rendelkező, de különböző típusú entitások megkülönböztetésére öröklődést alkalmazok. Ez azért is hasznos, mert a közös metódusokat nem kell osztályonként újradefiniálni ugyanazzal az implementációval.

Ilyen metódus például a koordináták lekérése, vagy az ütközésvizsgálat. Ebből az osztályból vannak származtatva azon osztályok, amelyek a megadott típusú entitásként kerülnek példányosításra. A játékban jelenleg az alábbi osztályok vannak származtatva, de ezek tetszés szerint bővíthetők:

- **Player:** játékos
- **Opponent:** ellenfél (többjátékos módban)
- **NPC:** számítógép által irányított karakter
- **Tile:** fal
- **Item:** tárgy (Emerald, Gem, Ruby, Gold, Start, Finish, FastPotion, SlowPotion)

A run metódus

Az osztály `run()` virtuális metódusának felüldefiniálásával megadható az entitás viselkedése a játékmotor futtatásakor. Alapvetően ez a metódus üres, tehát egy entitás alapvetően semmit sem csinál, statikusan áll egy helyben. Ilyenek például a falak, esetükben nincs szükség a mozgatásukra. A játékos (**Player**), vagy a tárgyak (**Item**) esetében azonban felül van definiálva, ugyanis ezek a játék során mozoghatnak. A játékost a felhasználó mozgatja, az NPC-ket a program irányítja, a tárgyak pedig a pályán lebegve forognak, illetve fel-le mozognak.

A HitBox struktúra

Az `entity.h` fájlban definiált **HitBox** struktúra egy adott entitást körülvevő doboz (**HitBox**) oldallapjainak (`front`, `back`, `right`, `left`, `top`, `bottom`) a középponttól vett távolságait tárolja. A hitboxokat az ütközésvizsgálathoz tartozó `checkCollision()` metódus fogja használni, amelyet a következő alfejezetben írok le részletesen.

A checkCollision metódus

A `checkCollision()` metódus ütközésvizsgálatot végez egy másik, paraméterként megadott entitással (**Entity***). Itt a `hitBox1` és a `hitBox2` az entitások po-

lignonhálójának kiterjedésével megegyező méretű hitboxok (téglatestek). Ellenőrizni kell, hogy a két doboznak van-e metszete, azaz legalább egy közös pontjuk. Ha létezik metszet, akkor a két entitás ütközik egymással. Ahhoz, hogy meg lehessen győződni a metszet létezéséről, többszintű elágazásokat alkalmazok. Ezek 3 szintű feltételek, amelyek mind a 3 tengelyen (x, y, z) leellenőrzik, hogy mely oldallapok (1, 2, 3, 4) értek egymásba.

A hitbox ütköző oldalai egy vektorba kerülnek (`collidingSides`), ugyanis több is lehet belőlük, de legfeljebb 2. Az elemszám alapján dől el a metódus visszatérési értéke és 3 fő eset kerül megkülönböztetésre:

- `collidingSides` elemszáma 0: nem történt ütközés, 0-val tér vissza
- `collidingSides` elemszáma 1: egy oldal ütközött, visszatér ezen oldal számával
- `collidingSides` elemszáma 2: két oldal ütközött, további eseteket kell megvizsgálni

Két oldal ütközése esetén meg kell vizsgálni, hogy az entitás melyik irányba mozgott az ütközés bekövetkezése előtt. Ezt az alapján lehet eldönteni, hogy a két oldal közül melyik oldal ér jobban a másik entitást körülvevő hitboxba. Ilyenkor ezen oldal számával tér vissza.

A `resolveCollision` metódus

A `resolveCollision()` metódussal az ütközésvizsgálat során detektált ütközést lehet feloldani egy másik, paraméterként megadott entitással. Egy while ciklust tartalmaz, amely egészen addig ismétlődik, amíg az ütközés létezése fennáll a két entitás között. Ezt a `checkCollision()` metódussal ellenőrzöm, amely visszatér az entitást körülvevő téglatestnek azon oldalának számával (1, 2, 3, 4), amellyel ütközött. Ha 0-val tér vissza, akkor nem történt ütközés. Az algoritmus a kapott szám alapján dönti el, hogy melyik irányba kell elmozgatni az entitást ahhoz, hogy a téglatesteknek ne legyen metszetük. De emellett figyelembe veszi azt is, hogy az entitás melyik tengelyen mozgott az ütközés előtt és a sebességével (`xVel`, `zVel`) megegyező sebességgel mozgatja vissza az irányával ellentétes irányba.

3.2.4. A Position struktúra

A `Position` struktúra háromdimenziós koordinátákat (`x`, `y`, `z`), valamint egy irányt tárol fokokban (`angleY`). Leggyakrabban a pályán elhelyezkedő entitások helyzetének tárolására használom, de emellett a grafikus felületen megjelenő vezérlő-elemek (például gombok, szövegek) pozícióját is ennek felhasználásával adom meg. Ezáltal nem szükséges minden egyes paramétert külön felsorolni, amikor például paraméterként adok át koordinátákat.

3.2.5. A Map osztály

A `Map` osztályban elsősorban a pályát felépítő entitásokat tárolom. Erre a célra egy listát (`std::list`) használok, amely entitásokra mutató pointereket (`Entity*`) tartalmaz.

A loadEntities metódus

Az entitások betöltését a `loadEntities()` metódussal végzem, amely a paraméterként megkapott stringstream-en (`std::stringstream`) végigiterálva példányosítja a megfelelő típusú entitásokat (például `Player`, vagy `Tile`). A lista betöltése történhet egy szekvenciális inputfájlból, vagy karaktersorozatból is, amikor többjátékos módban a kliens fogadja a szerver által küldött pályát. A lista a következő paramétereket tárolja egy adott sorban:

- `type`: entitás típusa
- `x`: x koordináta
- `y`: y koordináta
- `z`: z koordináta
- `angle`: irány fokokban

A metódusban emellett meghatározásra kerülnek a pálya legkisebb (`startX`, `startZ`) és legnagyobb (`endX`, `endZ`) koordinátái az x és z tengelyen, amelyeket két-két egyszerű minimum és maximum kereséssel határozok meg. Ezekre a koordinátákra majd később a pálya méretének meghatározásában lesz szükség.

A `saveState` metódus

A `saveState()` metódussal a pálya aktuális állapota menthető el a paraméterként megadott fájlba. Ez az állapot tartalmazza az entitások helyzetét, valamint az aktuálisan használt fal és skybox textúrák fájlneveit.

A `generateTileMeshes` metódus

A `generateTileMeshes()` metódusban implementált algoritmussal a falakhoz egyedi poligonhálókat generálok. A falak tulajdonképpen kockák, amelyek egymás mellé rakva egy hosszabb falat alkotnak, így az algoritmus figyelembe veszi, hogy mely falak kerülnek egymás mellé. A közvetlenül egymást követő falakat "egybeépíti", megszünteti a közöttük lévő oldallapokat, ezáltal ezen falak egy nagyobb, téglatest alakú falként jelennek meg a pályán.

3.2.6. A Camera osztály

A `Camera` osztállyal a kamera pozicionálását és mozgatását végzem polárkoordináta-rendszerben. A `yaw` (θ) és `pitch` (ϕ) értékeket a privát adattagokban tárolom lebegőpontos számként. A `yaw` értéke a kamera oldalirányú elfordulását az Y tengelyen, a `pitch` pedig a döntését határozza meg a Z tengelyen fokokban. Az osztály `getPosition()` metódusával lekérhető a kamera koordinátavektora derékszögű koordináta-rendszerben. Másik főbb metódus az `update()`, ami frissíti a θ , ϕ értékeit a megszorításoknak megfelelően ($0 \leq \theta < 360$, $-90 < \phi < 90$). A kamera koordinátáit tartalmazó vektor egy (x, y, z) valós számhármassal, amely a következő értékeket tartalmazza:

$$\begin{pmatrix} \cos\left(\phi \cdot \frac{\pi}{180}\right) \cdot \sin\left(\theta \cdot \frac{\pi}{180}\right) \\ \sin\left(\phi \cdot \frac{\pi}{180}\right) \\ \cos\left(\phi \cdot \frac{\pi}{180}\right) \cdot \cos\left(\theta \cdot \frac{\pi}{180}\right) \end{pmatrix} \in \mathbb{R}^3 \quad (3.1)$$

A `rotateTo` metódus

A `rotateTo()` metódussal a paraméterként megadott szögbe fordítható a kamera fokozatosan. Erre a menüben van szükség, amikor a program a labirintusjáték

bemutatásaként automatikusan végighalad egy véletlenszerűen generált pályán és irányváltoztatáskor a haladási irányba fordítja a kamerát.

3.2.7. A Client osztály

A `Client` osztály a hálózati kommunikációs műveleteket végzi kliensoldalon. A `CppThread` [3] absztrakt osztály leszármazottja. Példányosítása után először manuálisan meghívásra kerül a `start()` metódusa, amivel elindul egy külön szálon, majd a `connectToServer()` metódusa, amellyel csatlakozik a szerverhez. Paraméterként a szerver IP címét és a portszámát kell megadni. Sikeres csatlakozás esetén az osztály `connected` adattagját igaz értékre állítom. Ezután a `sendMessage()` és `receiveMessages()` metódusokkal lehet küldeni, illetve fogadni üzeneteket.

Annak biztosításához, hogy egy üzenet se vesszen el, az osztályban üzenetsort (`messageQueue`) használok az üzenetek fogadására, amelyben pontosan olyan sorrendben vannak elhelyezve az üzenetek, ahogyan a szerver küldte. Az osztályban definiálva van egy `DEBUG_MODE` konstans, melynek igazra állításával a kliens minden hálózati tevékenységet (például küldött, fogadott üzeneteket) a konzolra ír. A `MESSAGE_BUFFER_SIZE` konstanssal pedig az üzenetek pufferméretét lehet állítani, azaz, hogy egy adott TCP csomagban mekkora legyen az üzenet mérete.

A run metódus

Az osztály `run()` metódusa a kliensprogram futtatását végzi egy külön szálon, garantálva a párhuzamos futást. Az üzenetek fogadása egy blokkoló művelet, azaz csak akkor lép tovább, ha az üzenetet fogadott. Ahhoz, hogy a játékmotor megfelelően tudjon futni, miközben a kliens fogadja a beérkező üzeneteket a szervertől, a két folyamatot egymással párhuzamosan kell futtatni. Így el tudom kerülni, hogy maga a játék is blokkolódjon.

A metódusban egy ciklus található, amely egészen addig fut, amíg a `connected` adattag igaz értékű. A ciklusmagban fogadom az üzeneteket a `receiveMessages()` metódussal 100 milliszekundumos időtúllépéssel. Ez azt jelenti, hogy a fogadási művelet maximum csak a megadott időtúllépésig blokkol, majd továbblép, így elkerülhető, hogy a folyamat "beragadjon", ha már bontásra került a szerverrel való kapcsolat.

Ezután egy másik ciklus következik, amely az üzenetsor kiürüléséig ismétlődik. Itt a beérkezett üzeneteket egyesével lekezelem a `handleMessage()` metódus segítségével.

A `sendMessage` metódus

A `sendMessage()` metódussal üzenetek küldhetők a szerver felé. Paraméterként az elküldeni kívánt üzenetet fogja kapni, amely `string` típusú. A küldés előtt az üzenet végéhez hozzáfűz egy pontosvesszőt. Ez fogja jelezni a szerver számára az üzenet végét. A metódus ellenőrzi, hogy sikeres volt-e a küldés, ellenkező esetben kiírja a hiba nevét és okát a konzolra (`SDLNet_TCP_Send()`).

A `receiveMessages` metódus

A `receiveMessages()` metódus használatával a szerver által küldött üzeneteket lehet fogadni. Paraméterként megadható az időtúllépés mértéke milliszekundumokban (`timeout`), ezáltal az üzenetek fogadásához tartozó művelet (`SDLNet_TCP_Recv()`) legfeljebb a megadott ideig fog blokkolni. Ha az időtúllépés értéke 0, akkor a fogadás mindig a következő üzenet beérkezéséig blokkol.

Fogadáskor megállapításra kerül az üzenet hossza (`length`). Ha az üzenet hossza nem pozitív, akkor az azt jelenti, hogy a szerverrel való kapcsolat megszakadt és bontani kell vele a kapcsolatot (`disconnectFromServer()`). Ellenkező esetben a fogadás sikeresnek tekinthető. Sikeres fogadás esetén az üzenetet rövidebb üzenetekre darabolom fel a pontosvesszők mentén, majd ezeket a beérkezési sorrendben egy üzenetsorba helyezem.

A `popMessage` metódus

A `popMessage()` metódussal az elsőként megkapott üzenetet lehet levenni az üzenetsorról. A metódus működése azonos a sor adatszerkezet `pop()` műveletével, amely eltávolítja, majd visszaadja a sor soron következő elemét.

Az `init` metódus

A `Client` osztály `init()` metódusával a többjátékos módhoz szükséges adattagokat inicializálom. Először fogadom a szervertől a pályaadatokat, amelyet base64-ben dekódolok, majd gzip-el kitömörítetek. Ezután a kapott adatok alapján inicializálom a

játékosokat tartalmazó JSON objektumot, majd frissítem a játékosok pontszámait. Végül eltárolom a játékos azonosítószámát, amely alapján később azonosítani lehet a játékost (`playerId`).

A `pushChatMessage` metódus

A `pushChatMessage()` metódus segítségével egy paraméterként megadott szöveggel rendelkező üzenet írható a többjátékos módban megjelenő chat ablakba. Egy `(std::string, int)` párt helyez a `chatMessages` vektorba, amely a chat ablakban megjelenő üzeneteket tárolja. A pár első eleme az üzenet szövege, a második pedig a lejárat ideje. A lejárat idő a fogadástól számított 5 másodperc, így az ennél régebben küldött üzenetek már nem fognak megjelenni a chat ablakban.

A `handleMessage` metódus

A `Client` osztály `handleMessage()` metódusában a kliens által fogadott üzenetek kezelését végzem. A szerverprogramhoz hasonlóan, az üzenetek itt is rendelkeznek egy `action` kulccsal, amely meghatározza a végezni kívánt műveletet (például `join-game`, vagy `pick-up-item`).

A `loadOpponents` metódus

A `loadOpponents()` metódusban a többjátékos módban megjelenő többi játékos, azaz az ellenfelek pályára való betöltését végzem. Egy `for` ciklussal végigmegyem a szerver által küldött játékosok adatain (`players`), majd a `Map` osztály `addOpponent()` metódusának használatával létrehozom őket a pályán.

A `sendPlayerPosition` metódus

A `sendPlayerPosition()` metódussal elküldöm a kliens által kezelt játékos koordinátáit (`position`) és irányát (`angle`) a szervernek, amelyet az továbbít a többi kliens felé. Ez csak abban az esetben történik, ha a játékos helyzete változott az előző helyzetéhez képest. Ellenkező esetben nem kerülnek elküldésre az adatok.

3.2.8. Az Editor osztály

Az `Editor` osztályban a pályaszerkesztő funkciók működését implementáltam. A konstruktora paraméterként megkapja a `DrawUtils` segédosztály egy példányát, majd inicializálok minden, a használatához szükséges adattagot. Mivel az osztály egy különálló szoftverkomponens, ezért az eseményeket is külön kezeli.

A `run` metódus

Az osztály `run()` metódusában egy ciklus található, amely egészen addig ismétlődik, amíg a pályaszerkesztő futtatása során valamilyen esemény kilépést nem vált ki. A ciklusmagban vannak a pálya és a felhasználói felület kirajzolásához tartozó (`drawMap()`, `drawGui()`), valamint az eseménykezelést végző metódusok (`handleEvents()`, `handleMouse()`). Továbbá tartalmaz egy `switch` esetszétválasztást, amely a `handleEvents()` által megkapott esemény száma alapján valamilyen műveletet hajt végre. Ezen események a következők lehetnek:

- 1: új pálya létrehozása
- 2: labirintus generálása a megadott paraméterek alapján
- 3: pálya megnyitása
- 4: pálya mentése
- 5: kilépés

A pálya mentése esetén leellenőrzöm, hogy a pálya tartalmaz-e bejáratot (`start`), illetve kijáratot (`finish`), melyek megléte nélkül a program nem engedi elmenteni a pályát. Ennek ellenőrzése úgy történik, hogy a program megvizsgálja a `start` és `finish` blokkokra mutató pointereket és ha valamelyik nullpointer, akkor a `saveMap()` metódus meghívása helyett egy hibaüzenetet jelenik meg.

A pályaszerkesztővel szerkesztett pályák blokkokból épülnek fel, minden ilyen bloknak van egy típusa (`type`), koordinátái (`x`, `y`, `z`), valamint egy iránya (`angle`). A pálya mentésekor a blokkok adatait tartalmazó rekordokat kiírom egy `map` kiterjesztésű pályafájlba, amelyet a játékmotor, vagy a szerverprogram később be tud tölteni. Azt a struktúrát, ami egy blokkot reprezentál, a következőképpen definiálok:

```
struct Block {
    std::string type;
    float x, y, z, angle;

    Block(const std::string& type, float x, float y, float z, float
        angle) {
        ...
    }
};
```

3.1. forráskód. A Block struktúra

A drawBlockTo metódus

Az Editor osztály `drawBlockTo()` metódusával a paraméterként megadott pozícióra (x, y) rajzolom a megadott típusú blokk textúráját. A falak textúráját mindig a kiválasztott faltextúra (`selectedTileTexture`) szerint választom ki. A `Start` mező esetén egy narancssárga nyilat rajzolok, amely abba az irányba mutat, ahova a játékos alapértelmezetten néz a játék indulásakor.

A transformMapPosition metódus

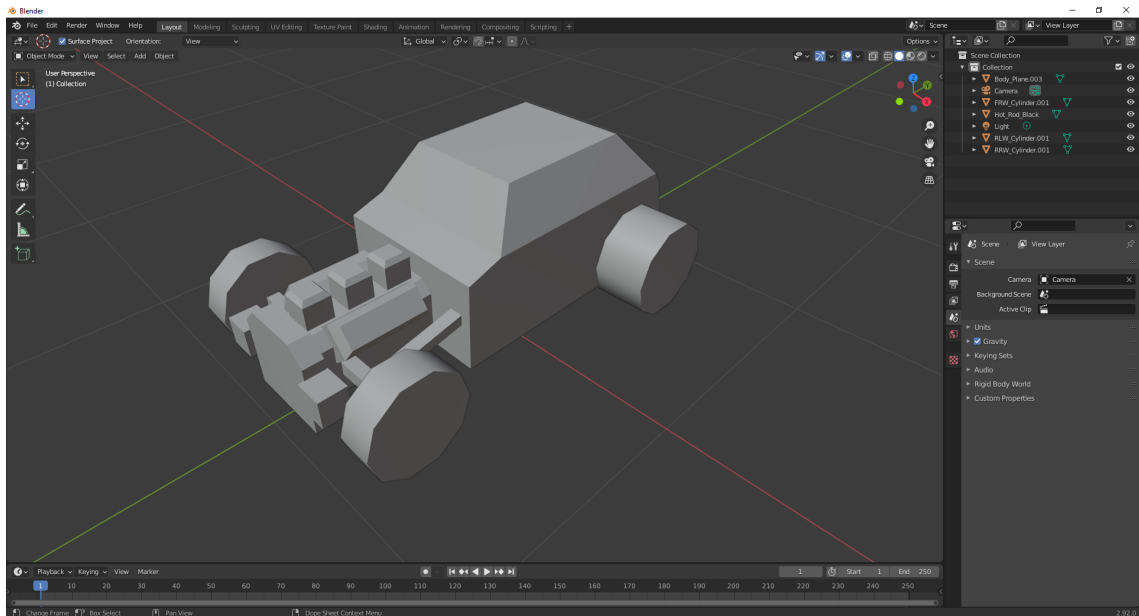
A `transformMapPosition()` metódussal egy blokk relatív koordinátái alakíthatók át a kamerapozíció (`cameraX`, `cameraY`) figyelembevételével abszolút koordinátákká. Ezzel meghatározható, hogy a képernyő pontosan mely részére kell kirajzolni egy adott blokkot.

A drawMap metódus

A `drawMap()` metódusban a pályaszerkesztővel készített kétdimenziós pályát rajzolom a képernyőre. Egy `for` ciklussal végigiterálok a pályát felépítő blokkokon. A ciklusmagban először a `transformMapPosition()` metódus alkalmazásával előállítom a blokk relatív koordinátáit a kamerapozícióhoz képest, ezután a `drawBlockTo()` metódusnak paraméterként adva a blokk pozícióját (`position`), típusát (`type`) és irányát (`angle`) a képernyőre rajzolom. A kirajzoláskor mindig csak a kamera által látható blokkokat rajzolom ki. Így nem okoz lassulást a nagy méretű pályák blokkjainak kirajzolása.

3.2.9. A Mesh osztály

A Mesh osztály egy modellt (poligonhálót) reprezentál, amely egy vagy több entitáshoz kerül hozzárendelésre. Az entitás modelljével tulajdonképpen azt adom meg, hogy hogyan fog megjelenni a háromdimenziós térben.



3.3. ábra. Egy jármű poligonhálója szerkesztés közben

Általában egy szekvenciális inputfájlból töltöm be a poligonhálót a `loadOBJ()` metódus segítségével, de lehetőség van a programon belül is létrehozni az `addVertex()`, illetve az `addIndex()` metódusok megfelelő alkalmazásával. Az osztály csúcs struktúrákat (Vertex), indexeket, valamint egy textúrát tárol. Egy ilyen Vertex struktúra a következőképpen néz ki:

```
struct Vertex {
    glm::vec3 position;
    glm::vec3 normal;
    glm::vec2 texCoord;
};
```

3.2. forráskód. A Vertex struktúra

Az init metódus

Az osztály `init()` metódusával inicializálódik az osztályban tárolt Vertex Array Object (VAO), Vertex Buffer Object (VBO) és az indexpuffer, miután létrejöttek

a csúcsok, illetve az indexek. Az OpenGL a VAO-ból fogja a képernyőre rajzolni a poligonhálót a `glDrawElements()` procedúra meghívásával. A VAO, VBO és indexpuffer inicializálása a következő lépésekből áll:

1. VAO, VBO, indexpuffer generálása, memóriaterületének allokálása és neveik eltárolása az `m_vaoID`, `m_vboID`, `m_ibID` változókbán (`glGenBuffers()`)
2. Az `m_vaoID`, `m_vboID` nevű VAO és VBO csatlakoztatása (`glBindVertexArray()`, `glBindBuffer()`)
3. A `vertices` vektor által tárolt `Vertex` típusú rekordok másolása a VBO-ba (`glBufferData()`)
4. A 0-ás indexű attribútum használatának engedélyezése a VAO-ban (`glEnableVertexAttribArray()`)
5. A koordináták (`position`) a VAO 0-ás indexű attribútumába kerülnek (`glVertexAttribPointer()`)
6. A 1-es indexű attribútum használatának engedélyezése a VAO-ban (`glEnableVertexAttribArray()`)
7. A normálvektorok (normálok) a VAO 1-es indexű attribútumába kerülnek `glm::vec3`-al eltolva (`glVertexAttribPointer()`)
8. A 2-es indexű attribútum használatának engedélyezése a VAO-ban (`glEnableVertexAttribArray()`)
9. A textúrákoordináták a VAO 2-es indexű attribútumába kerülnek, $(2 * \text{glm::vec3})$ -al eltolva (`glVertexAttribPointer()`)
10. Az `m_ibID` nevű indexpuffer csatlakoztatása (`glBindBuffer()`)
11. Az `indices` vektor által tárolt `GLuint` típusú indexek betöltése az indexpufferbe (`glBufferData()`)
12. VAO, VBO, indexpuffer lecsatlakoztatása (`glBind` előtagú hívások, 0-ás paraméterrel)

Egy poligonháló kirajzolása

A poligonhálók kirajzolását a `Game` osztály `drawMesh()` metódusával végzem perspektivikus vetítés esetén. A metódus paraméterként megkapja a kirajzolni kívánt modell memóriacímét (`Mesh*`), koordinátáit (`position`), valamint a használni kívánt shader azonosítóját (`m_loc`). A kirajzolás a következő lépésekből áll:

1. Kiszámítom a modellmátrixot (`model`) a transzformációk (mozgatás, forgatás) alkalmazásával, majd veszem ennek inverzét és transzponáltját (`modelIT`).
2. Előállítom az MVP (Model View Projection) mátrixot a projekciós- (`projection`), nézet- (`view`) és a modellmátrixok (`model`) szorzataként.
3. Uniform változóként betöltöm a shaderbe a 3 mátrixot (`model`, `modelIT`, `MVP`), valamint a játékos pozícióját (`playerPos`) és a renderelési távolság mértékét (`renderDistance`).
4. Lekérdezem a poligonháló textúráját az objektum `getTexture()` metódusával, majd ezt is betöltöm a shaderbe uniform változóként.
5. Csatlakoztatom (bindolom) a poligonháló VAO-ját (Vertex Array Object) a `glBindVertexArray()` hívással.
6. Végül a képernyőre rajzolom a poligonhálót a `glDrawElements()` használatával.

Egy indexszel mindig egy adott csúcsra lehet hivatkozni, így a csúcsok indexelhetők. Ez azért hasznos, mert nem szükséges többször ugyanazt a csúcsot definiálni egy poligonháló létrehozásakor, elég egy már korábban létrehozott csúcsra hivatkozni. Emellett segítségükkel megadható a csúcsok kirajzolási sorrendje. Fontos tudni, OpenGL-ben csúcsok kirajzolását az óramutató járásával ellentétes irányban kell végezni, hogy előrefelé néző arcelemeket (`faces`) lehessen kapni.

A `loadOBJ` metódus

A metódus alkalmazásával poligonhálót lehet betölteni fájlból a paraméterként megadott elérési útvonalról. A fájl `obj` kiterjesztésű, amely a Wavefront OBJ [4]

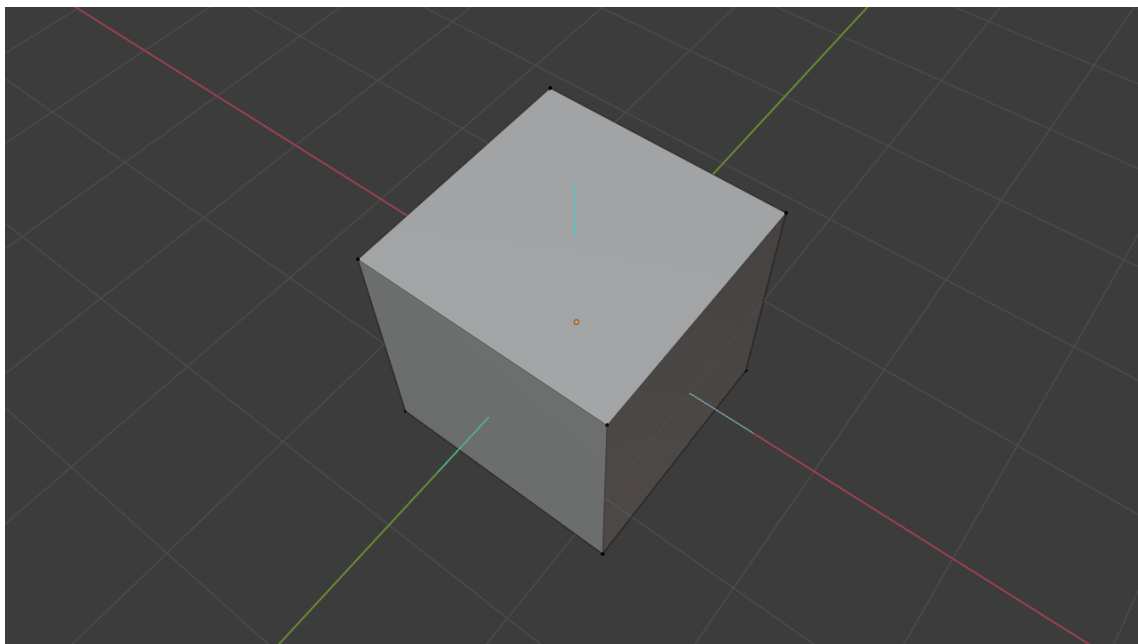
szabványának megfelelő, de kiegészítésként tartalmazza a használni kívánt textúra elérési útvonalát is. Erre a `texture` kulcsszóval hivatkozom.

A fájl egy adott sora a következő paramétereket tartalmazhatja:

- `v`: csúcs koordinátái (x, y, z)
- `vn`: normálvektor koordinátái (x, y, z)
- `vt`: textúrákoordináták (u, v)
- `f`: arcelem (csúcsindex1/textúraindex1/normálindex1 ...)
- `texture`: textúra elérési útvonala

Az arcelemek (faces) tulajdonképpen a modellt alkotó lapocskák. Minden ilyen kis lapnak egy 3-nál több szögű sokszögnek kell lennie ahhoz, hogy ki lehessen rajzolni egy síkként a háromdimenziós térben. A játékmotor erre a célra háromszögeket alkalmaz, így három index-hármasnak kell szerepelnie egy arcelem meghatározásához.

Minden más kulcsszóval kezdődő sor ignorálásra kerül beolvasáskor, azaz lehetőség van kommentek írására is, a `#` karakter kiírásával.



3.4. ábra. Egy kocka lapjainak normálvektorai

3.2.10. A Dimension struktúra

A `mesh.h` fájlban definiált `Dimension` struktúra egy adott poligonháló kiterjedését tárolja. Betöltéskor a `loadOBJ()` metódus meghatározza a poligonháló szélességét (`width`), magasságát (`height`), mélységét (`depth`), amelyeket ebben a struktúrában eltárol. A `Mesh` osztály rendelkezik egy `getDimensions()` metódussal, amellyel lekérhető annak kiterjedése. Erre általában az ütközésvizsgálat esetén van szükség, amikor ismerni kell a modellek méreteit, így meghatározható, hogy egy adott entitáshoz tartozó poligonháló mikor ütközött például fallal.

3.2.11. A Maze osztály

A `Maze` osztály a labirintusgenerálásért felelős. Mind a játékprogram és a szerverprogram ugyanazt az implementációt használja. Segítségével tetszőleges méretű labirintusok generálhatók a bejárat, illetve a kijárat koordinátáinak megadásával. A labirintus reprezentációjának tárolására egy $n \times m$ -es mátrixot használok, amely cellákra mutató pointereket tartalmaz (`Cell*`). A mátrix mérete mindig a példányosításkor megadott szélesség és magasság paraméterektől függ. A konstruktor a mátrix inicializálása után meghívja a `randomizeDfs()` metódust, amely elindítja a labirintusgenerálási folyamatot.

A Cell osztály

A labirintust felépítő cellákat a `Cell` osztállyal reprezentálom. Adattagjaiban a következő értékeket tárolom:

- `x`, `y`: a cella kétdimenziós pozíciója
- `type`: a cella típusa
 - 0: közönséges cella
 - 1: kijárat
 - 2: bejárat
 - 3: tárgyat tartalmazó cella
 - 4: NPC-t tartalmazó cella

- **visited**: a cellát látogatottként jelölte meg az algoritmus
- **topWall**, **bottomWall**, **leftWall**, **rightWall**: a cellát körülvevő falak állapota
 - igaz érték esetén létezik, ellenkező esetben nem létezik a fal
- **finishPath**: a cella eleme a kijáráshoz vezető útvonalnak

Az algoritmus

A labirintusgenerálási algoritmus a mélységi bejárás (depth-first search) alapul, amely egy gráfelméleti keresőalgoritmus. A mélységi keresés egy változatát a 19. században a francia matematikus, Charles Pierre Trémaux a labirintusok megoldásának stratégiájaként vizsgálta meg [5], tőle származik az alapelve. Az algoritmus alapvetően rekurzív, de megvalósítható iteratív módon is. Én az utóbbi módon implementálom. Rekúzió esetén ugyanis a végrehajtási verem mérete korlátozott és az algoritmus egy bizonyos mélység után leáll, ami hibát eredményez (stack overflow). Ennek a problémának a kiküszöbölésére iteratívan kell megírni az algoritmust, úgy, hogy a végrehajtási verem egy verem adatszerkezettel van helyettesítve (`std::stack`). Ennek a veremnek már jóval nagyobb a kapacitása, így nem kell arra számítani, hogy az algoritmus például egy 100×100 -as labirintus generálása esetén leáll. Az algoritmus implementációját a 3.4.1 pontban található struktogram mutatja be.

Első lépésként kiválasztom a paraméterek alapján (**startX**, **startY**) a kezdőcellát (**initialCell**), majd megjelölöm látogatottként és bejáratként. Ezután berakom egy verembe, majd elindul egy ciklus, amely a verem kiürüléséig fog ismétlődni.

A ciklus először kiveszi a veremből az aktuális cellát (**currentCell**). Ezután eldönti, hogy kell-e tárgyat elhelyezni a cellában. Ha a cella nem bejárat, akkor 1/10-ed esélye van annak, hogy a cellába valamilyen tárgy, vagy NPC kerül. Így megoldható, hogy a labirintusban ritkán és szétszórtan jelenjenek meg különböző tárgyak és NPC-k. Következő lépésként egy vektorba gyűjti azon cellaszomszédokat, amelyeket még nem látogatott meg a `getUnvisitedNeighbors()` metódus használatával. Ha a vektor nem üres, azaz vannak ilyen szomszédok, akkor véletlenszerűen választ közülük egyet, amelyet szintén látogatottként jelöl meg. Ezután eltávolítja

a kiválasztott és az aktuális cella közötti falat a `removeWallBetween()` metódussal, majd mindkét cellát a verembe helyezi.

Miután a ciklus lefutott, meghívásra kerül a `backtrackIter()` metódus a `(startX, startY, endX, endY)` paraméterekkel, amely a bejárat és a kijárat közötti utat fogja legenerálni a labirintusban. Az algoritmus a mélységi bejárás alapuló visszalépéses keresés (backtracking) [6] módszerét alkalmazza (3.4.2 pont).

A `generateEntities` metódus

A `generateEntities()` metódussal a labirintust tároló mátrix (`mazeCells`) cellái alapján entitások listáját lehet generálni, amely később elmenthető, vagy a klienseknek továbbítható. Ezen listát nem csak a cellák által tárolt elemek alapján építi fel, hanem a cellákat körülvevő falakat (felső, alsó, bal, jobb) is figyelembe veszi.

A `save` metódus

Az osztály `save()` metódusával a paraméterként megadott fájl névvel elmenthető a generált labirintus. A fájlba az entitások listája kerül, amely soronként tartalmazni fogja az adott entitás típusát, pozícióját, illetve irányát.

A `getData` metódus

A `Maze` osztály `getData()` metódusával a generált labirintus reprezentációját lehet lekérni `std::stringstream`-ben, amely az entitások listáját fogja tartalmazni. Ezen a listán iterál végig a `Map` osztály `loadEntities()`, illetve az `Editor` osztály `loadMap()` metódusa, amikor betöltődnek a pálya adatai.

A `getSolutionPath` metódus

A `getSolutionPath()` metódussal az eltárolt, megoldáshoz vezető útvonalat lehet lekérni `std::list` listában. Az útvonal elemei olyan `(x,y)` párok, amelyek a kezdőcellától kezdve a kijáratig tartalmazzák az útvonalhoz tartozó cellák koordinátáit egymást követő sorrendben. Az algoritmus a `backtrackIter()` futása során megoldásként megjelölt cellái közül elhelyezi listában a megfelelő cellákat. Ügyel arra, hogy mindig csak azon koordináták kerüljenek be a listába, ahol a játékosnak irányt kell változtatnia (például a sarkokon).

Az `isWallBetween` metódus

Az `isWallBetween()` metódus a paraméterként megadott két celláról eldönti, hogy van-e köztük fal, amely elválasztja őket. Legalább egy ilyen létező fal esetén igaz, ellenkező esetben hamis értékkel tér vissza.

A `getUnvisitedNeighbors` metódus

A `getUnvisitedNeighbors()` metódus az olyan cellaszomszédokat határozza meg a paraméterként megadott cellára, amelyeket az algoritmus korábban még nem látogatott meg, azaz a `visited` adattagjuk hamis volt. Visszatérési értéke egy cellákra mutató pointereket tartalmazó vektor (`std::vector<Cell*>`).

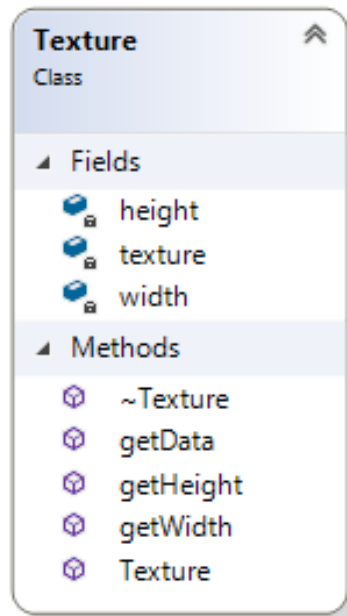
A `getUnvisitedAccessibleNeighbors` metódus

A `getUnvisitedAccessibleNeighbors()` metódus az olyan cellaszomszédokat határozza meg a paraméterként megadott cellára, amelyek még korábban nem voltak meglátogatva, valamint a megadott cellából elérhetőek, azaz nincs köztük fal, amely elválasztja őket. Visszatérési értéke egy cellákra mutató pointereket tartalmazó vektor (`std::vector<Cell*>`).

A `putItem` és `putNPC` metódusok

A `putItem()` metódussal egy tárgyat, a `putNPC()` metódus használatával pedig egy NPC-t lehet elhelyezni a paraméterként megadott cellában. A `putNPC()` metódusban ellenőrzöm az elhelyezett NPC-k számát és csak akkor helyezek el egy újabbat, ha a számuk (`totalNPCs`) nem haladja meg a `MAX_NPC_COUNT` konstans által definiált korlátot.

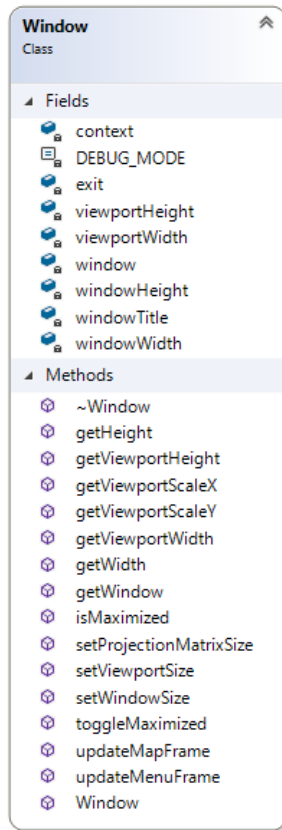
3.2.12. A Texture osztály



3.5. ábra. A Texture osztály osztálydiagramja

A **Texture** osztály különböző felületek, illetve poligonhálók által használt textúrákat reprezentálja, amely tulajdonképpen egy csomagoló (wrapper) osztály. Konstruktorának paraméterként a tárolni kívánt textúrát adom (**GLuint**), majd OpenGL-es hívásokkal lekérdezem a szélességét és a magasságát, amit az adattagjaiban eltárolok. Így később egy-egy egyszerű getter metódussal lekérhetők a textúra méretei. A tárolt textúra pedig a **getData()** segítségével kérhető le.

3.2.13. A Window osztály



3.6. ábra. A Window osztály osztálydiagramja

A Window osztály a játékmotor programablakát reprezentálja, illetve vezérli a megfelelő metódusok segítségével. Példányosításkor létrehozza az SDL-es programablakot (`SDL_Window*`) a megfelelő paraméterekkel, valamint az inicializálja OpenGL működéséhez szükséges környezetet (`SDL_GLContext`).

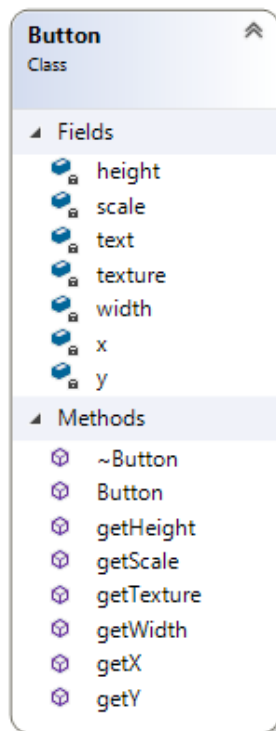
Az `updateMapFrame` metódus

Az `updateMapFrame()` metódus a képkockafrissítést végzi perspektivikus vetítés esetén (`glm::perspective`), azaz a háromdimenziós pálya kirajzolásakor. A kamera pozíciójától és beállításaitól függően beállítom a nézetmátrixot (`view`) a `glm::lookAt()` alkalmazásával. Ha a kamera 0-ás módban van belső nézetes (FPS), ha pedig 1-es módban van, akkor pedig hátulnézetes (TPS) megjelenítés kapható.

Az updateMenuFrame metódus

Az `updateMapFrame()` metódus szintén képkockafrissítést végez, azonban ortografikus vetítés esetén, például a menüben, ahol nincs szükség háromdimenziós megjelenítésre. Ilyenkor hagyományos kétdimenziós felületeket rajzolok ki a képernyőre.

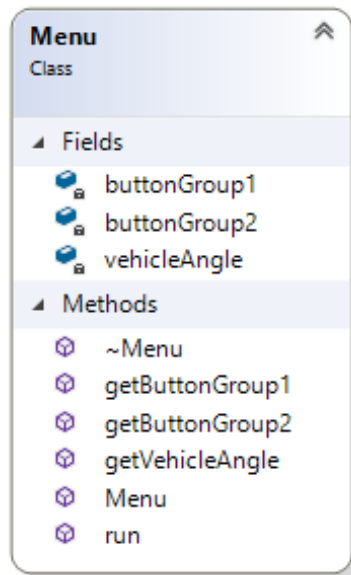
3.2.14. A Button osztály



3.7. ábra. A Button osztály osztálydiagramja

A **Button** osztály a felhasználói felületen (GUI) megjelenő gombokat reprezentálja. Ilyen gombok a játékmenüben jelennek meg, de tetszés szerint bárhol elhelyezhetők a játékban. Konstruktora megkapja a gombon megjeleníteni kívánt szöveget (`text`), a gomb pozícióját (`x`, `y`), méretét (`scale`), illetve a szöveg bittérképes betűkészletét (`font`s). Ezután a `renderText()` metódussal legenerál egy textúrát (`Texture*`), amit adattagként eltárol és egy getter metódussal lekérhető. Ezzel a textúrával rajzolom ki a gombot.

3.2.15. A Menu osztály



3.8. ábra. A Menu osztály osztálydiagramja

A `Menu` osztály a játék főmenüjét reprezentálja. Listákban tárolja a főmenü különböző részein megjelenő gombokat (`buttonGroup1`, `buttonGroup2`), amelyek gombokra mutató pointereket tartalmaznak (`Button*`). Konstruktorában feltölti a listákat ezekkel a gombokkal, mindegyiknek egyenként megadhatók a paraméterei, például a szövege, vagy a pozíciója.

3.2.16. Főprogram (main.cpp)

A `main.cpp` fájlban található a program fő belépési pontja, a `main()` függvény. Itt lényegében csak példányosításra kerül a játékmotort reprezentáló `Game` osztály, majd meghívódik a `run()` metódusa a futtatáshoz. Ezzel elindul a játékmotor, ami egészen addig fut, amíg valamilyen esemény kilépést nem vált ki, azaz a `quit` adattag igaz értéket nem vesz fel. Ekkor a `run()` metódus leáll, továbblép, majd 0-val tér vissza, jelezve az operációs rendszernek a sikeres lefutást.

3.2.17. Segédfüggvények (drawutils.h)

A `drawutils.h` fájl a `DrawUtils` segédosztályt tartalmazza, valamint olyan általános segédfüggvényeket deklarál, amelyek a játékmotorban lényegében bárhol felhasználhatók. Ezen metódusok, illetve függvények implementációja a `drawutils.cpp` fájlban található, működésük az alábbi táblázatban van kifejtve:

Név	Leírás
<i>drawTexture2D</i>	Kétdimenziós textúrát rajzol a képernyőre a megadott paraméterekkel (textúra, pozíció (x, y), skálázás (scale), méretek (width, height))
<i>drawRectangle</i>	Kétdimenziós téglalapot rajzol a képernyőre a megadott paraméterekkel (szín, pozíció (x, y), méretek (width, height))
<i>drawBackground2D</i>	Háttérképet rajzol ismétlődő mintázattal a paraméterként megadott textúrával
<i>drawText</i>	Szöveget rajzol a képernyőre a megadott paraméterekkel (pozíció (x, y), skálázás (scale))
<i>drawTextInput</i>	Szövegbeviteli mezőt rajzol a megadott paraméterekkel (üzenet, begépeltek szöveg)
<i>drawButtonGroup</i>	A paraméterként megadott gombok listáját rajzolja ki egyszerre a képernyőre
<i>loadImage</i>	A paraméterként megadott képfájlt betölti egy <code>SDL_Surface*</code> felületre
<i>createTextureFromSurface</i>	A paraméterként megadott <code>SDL_Surface*</code> típusú felületet textúrává alakítja
<i>createTextureFromImage</i>	A paraméterként megadott képfájlt beolvassa és textúrává alakítja
<i>renderText</i>	A megadott szövegű és betűtípusú szöveget textúrába rendereli (ennek működése a következő alfejezetben van részletezve)

3.1. táblázat. A segédfüggvények és metódusok leírása

A `renderText` segédfüggvény

A `renderText()` segédfüggvénnyel a megadott betűkészletű szöveget textúrába renderelem (rajzoló). Több képfájlból álló bittérképes betűkészletet alkalmazok, amely a játékban használt összes karaktert tartalmazza. Jelenleg az angol abc kis- és nagybetűit (a-Z), a számjegyeket (0-9), valamint az egyéb speciális karaktereket (felkiáltójel, pont, alulvonás stb.) használom. Mindegyik kategóriát külön képfájlban kell tárolni, erre egy példa a 3.9. ábrán látható.



3.9. ábra. A kisbetűs betűtípust tartalmazó képfájl

Első lépésként létrehozok egy üres felületet. Ezután egy `for` ciklussal végigiterálok a megadott szövegen, majd a soron következő karakternek megfelelő képet a felületre másolom. Számok és betűk esetén a karakter ASCII kódja határozza meg, hogy honnan, milyen pozícióról kell a képet lemásolni.

Második lépésként létrehozok egy újabb üres felületet, amelyre a végeredmény fog kerülni. Ennek szélessége az előző lépésben kirajzolt szöveg szélességétől függ. Ezzel lényegében "levágásra" kerül a fel nem használt terület, úgy, hogy csak a szükséges rész, azaz a szöveg maradjon meg. Ezt követően egy textúrát készítek a felületből a `createTextureFromSurface()` segédfüggvényt alkalmazva.

3.3. A szerverprogram (maze-server)

A szerverprogram fogadja és kezeli a többjátékos módhoz csatlakozó játékosokat (klienseket). Mivel több klienst kell egyszerre párhuzamosan kiszolgálni, ezért a program több szálon fut és minden egyes klienst egy-egy szál (worker) szolgál ki. A szerver a kommunikáció során szövegalapú, JSON [2] formátumot használ. A program egy konzolos alkalmazás, így nem rendelkezik saját grafikus felülettel (GUI), hanem induláskor egy konzolablak jelenik meg, ahonnan parancsokkal közvetlenül vezérelhető a működése.

3.3.1. A Server osztály

A **Server** osztály a szerverprogram fő részét képezi. Itt fogadom a klienseket, valamint a különböző üzeneteket is innen küldöm. Amikor példányosítom, megnyitok egy TCP alapú socketet a konfigurációs fájlban megadott portszámon. Ha minden hívás hibamentesen lefutott, akkor legenerálásra kerül egy új labirintus a `generateMap()` metódussal, amit a játékosok fognak megkapni, amikor csatlakoznak.

Miután az osztályt példányosítottam, meghívom a `start()` metódusát, amellyel meghívódik a `run()` metódusa, ezzel elindul a szerverprogram fő ciklusa egy külön szálon. Ez a ciklus egészen addig fut, amíg a szervert le nem állították, vagyis a `stop` adattag igaz értéket nem vesz fel. A ciklus az üzenetsoron lévő üzenetek kiküldését végzi (`handleMessageQueue()`), illetve fogadja az újonnan bejövő klienseket (`acceptConnection()`).

Ha sikeresen létrejött a klienssel való kapcsolat, akkor a **ServerThread** osztály példányosításával létrejön egy új szál, amin keresztül a klienssel lehet kommunikálni. Az osztály konstruktora paraméterként megkapja a **Server** objektumra mutató pointert, a fogadáskor létrejött socketet, valamint egy véletlenszerűen generált UUID-t, ami alapján azonosítható a kliens. Az UUID (Universally Unique Identifier) egy olyan véletlenszerűen generált hexadecimális formátumú azonosító, amely minden esetben egyedi lesz. Emellett ahhoz a `serverThreads` listához adom hozzá, amelyben számon vannak tartva a létrehozott szálak.

Üzenetküldés

A `broadcast` előtagú metódusok használatával különféle üzeneteket lehet küldeni (közvetíteni) az összes, szerverhez csatlakozó kliens felé. Minden ilyen üzenet tartalmazza a végezni kívánt műveletet (`action`) és a kapcsolódó paramétereket. Üzenetküldéskor az üzenet mindig egy üzenetsorba kerül.

Mivel ezt az üzenetsort egyszerre több szál is írhatja, ezért azt szálbiztossá kell tenni, azaz kölcsönös kizárást kell alkalmazni, hogy egy időben csak egy szál tudja írni. Ennek megvalósításához mutexet (`std::mutex`) használok. A mutex lényegében egy bináris szemafor, amely szabad jelzés esetén csak egy szálat enged tovább, a többit pedig blokkolja, amíg a továbbengedett szál nem végez és fel nem oldja a zárat.

A `setPlayerPosition` metódus

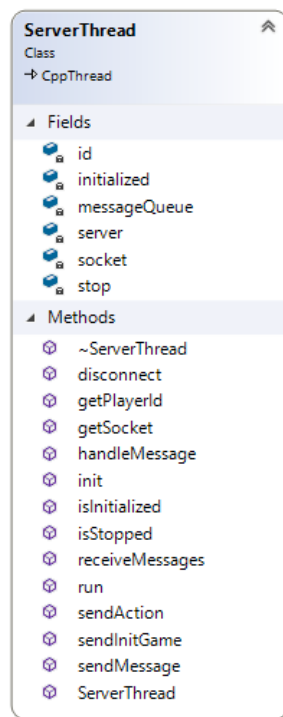
A `setPlayerPosition()` metódussal a paraméterként megadott azonosítóval rendelkező játékos pozícióját lehet beállítani a pályán. Ezen metódus általában a játékos mozgása esetén kerül meghívásra. Mivel egyszerre több szál is mozgathatja a játékosokat és írhatja a játékosok koordinátáit tartalmazó objektumot, ezért itt is (hasonlóan az üzenetküldéshez) kölcsönös kizárást alkalmazok.

A `checkCollision` metódus

A `checkCollision()` metódusban ütközésvizsgálatot végzek a paraméterként megadott azonosítóval (`id`) rendelkező játékoskal. Egy `for` ciklussal végigiterálok a pálya entitásain és ellenőrzöm, hogy a játékos pozíciója az entitás egy adott sugarú környezetében van-e. A vizsgálatkor figyelembe veszem az entitás típusát, amely alapján megállapítom, hogy milyen műveletet kell végezni ütközés esetén. Ilyen műveletek például, hogy valaki éppen elérte a célt, vagy felvett egy tárgyat. Ezen műveletekről mindig értesítem a többi játékos is, így például a célba érés esetén egy újonnan generált pályát kapnak a játékosok.

3.3.2. A ServerThread osztály

A `ServerThread` osztály egy workert reprezentál, amely az adott klienssel való kommunikációt végzi. Az osztályt a `CppThread` [3] absztrakt osztályból származtatom. Ennek segítségével az osztály példányosítása után a `start()` metódusának meghívásával egy új szál (`std::thread`) jön létre, amelyen elindul az osztály `run()` metódusa. Ezen metódus egy while ciklust tartalmaz, amely egészen addig ismétlődik, amíg a `stop` adattag igaz értéket nem vesz fel. A ciklus leállása után bontásra kerül a klienssel való kapcsolat, majd a szál kilép.



3.10. ábra. A `ServerThread` osztály osztálydiagramja

A `run` metódus

Az osztály `run()` metódusa a worker futtatását végzi, amely az általa kezelt klienst szolgálja ki. Először fogad a játékostól egy üzenetet, amely tartalmazza a nevét és az általa kiválasztott járművet, majd megnöveli a játékosok számlálóját 1-el és inicializálja a játékos paramétereit (név, jármű, pontszám, pozíció). Ezt követően a `sendInitGame()` metódussal elküldi a játékosnak a szerver által generált azonosítóját, a játékosok helyzetét, pontszámukat, valamint az aktuálisan betöltött pálya adatait `gzip` [7] tömörítéssel és `base64` [8] kódolással. Ezután értesíti a

többi játékost az új játékos csatlakozásáról a `broadcastJoinGame()` meghívásával, majd elindít egy újabb ciklust. Ebben a ciklusban folyamatosan fogadja a kliens által küldött üzeneteket egy üzenetsorban, amelyeket a `handleMessage()` metódussal a beérkezési sorrendben egymás után lekezel. Minden egyes üzenet tartalmaz egy `action` kulcsot, amely meghatározza a kliens által végezni kívánt műveletet. Ezen művelet lekezelésre fog kerülni a `handleMessage()` metódus által.

A `handleMessage` metódus

A `handleMessage()` metódussal a kliens által küldött üzeneteket lehet értelmezni és kezelni. Ilyen üzenetek például a játékos mozgatása (`move-player`), vagy a chat üzenet küldése (`chat-message`). Általában minden beérkező műveletet továbbküldök a többi játékosnak valamely `broadcast` előtagú metódus meghívásával, így a többiek is látják egy másik játékos mozgását, vagy az általa küldött chat üzeneteket.

A mozgatás esetén működik egyfajta csalásmegelőzés is, azaz nincs megengedve, hogy 2 egységnél nagyobb távolságot lehessen megtenni az aktuális pozíciótól. Így megelőzhető, hogy valaki a pálya egyik végéből a másik végébe "teleportáljon".

Az `init` metódus

A `ServerThread` osztály `init()` metódusával az osztály által kezelt játékos paramétereit inicializálom fel a szerverprogramban. Első lépésként fogadom a kliens által küldött első üzenetet, amely tartalmazza a játékos nevét, valamint az általa választott jármű típusát. Ha a kliens nem küldi el az első üzenetét, akkor bontom vele a kapcsolatot. Ezután ellenőrzöm, hogy a szerveren létezik-e még szabad hely, azaz, hogy a szerverhez csatlakozott kliensek száma nem haladja-e meg a konfigurációs fájlban megadott maximális slotok számát (`maxSlots`).

Végül utolsó lépésként azt is ellenőrzöm, hogy a csatlakozni kívánó játékos neve nem egyezik-e meg valamelyik korábban csatlakozott játékos nevével. Ha a kliens minden validációnak megfelelt, akkor az általa kezelt játékos adatai hozzáadásra kerülnek a szerverprogram által kezelt játékosok listájához.

3.3.3. A Map osztály

A szerverprogram - a játékprogramhoz hasonlóan - szintén rendelkezik egy Map osztállyal, amely az aktuálisan betöltött pályát reprezentálja és a pályán lévő entitásokat tárolja. Az osztály `loadEntities()` metódusában a paraméterként megkapott `stringstream`-en végigiterálva létrehozom a megfelelő típusú entitásokat. A `setSkyboxTexture()` és `setTileTexture()` metódusokkal beállítható a pályán használni kívánt skybox-, illetve faltextúra. Ezeket alapvetően véletlenszerűen állítom be a konstruktorban, amikor létrejön az osztálypéldány.

3.3.4. Az Entity osztály

Az `Entity` absztrakt osztály a pályán elhelyezkedő entitásokat reprezentálja. Itt jóval egyszerűbb az implementációja, mint a kliensprogram esetén. Csak a használt entitástípusokat származtattam, amelyek az alapvető paraméterekkel (típusnév, pozíció, irány) rendelkeznek. Az entitások típusának megkülönböztetésére az ütközésvizsgálat miatt van szükség, így tudom eldönteni, hogy például a játékos a célba ért be, vagy éppen tárgyat vett fel.

3.3.5. A Config osztály

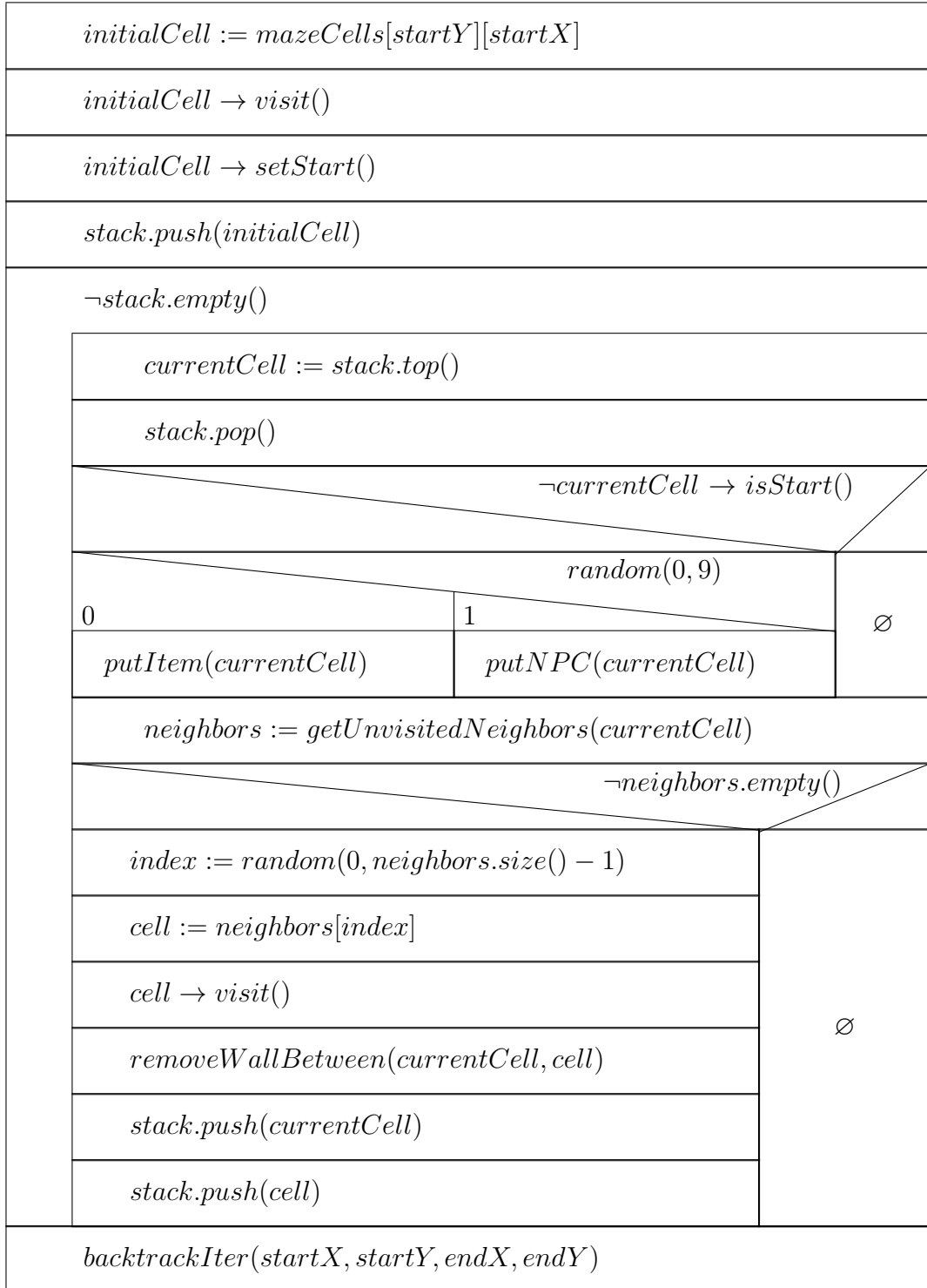
A `Config` osztály a szerverprogram konfigurációját tárolja. Implementációja hasonló, mint a játékmotor esetén, azonban az `init()` metódusában eltérő alapértelmezett beállításokat definiálok. Az alapértelmezett beállítások tartalmazzák a szerver portszámát (`port`), a maximális helyek számát (`maxSlots`), a pályafájl nevét (`mapFile`), valamint a generált labirintus méreteit (`width`, `height`).

3.4. Algoritmusok

3.4.1. A randomizeDfs algoritmus

A `randomizeDfs` egy mélységi bejáráson alapuló gráfelméleti algoritmus, amely a játékban a labirintusok generálását végzi. Az algoritmus verem adatszerkezettel megvalósított iteratív implementációját az alábbi struktogram szemlélteti:

randomizeDfs(*startX* : $\mathbb{Z}$, *startY* : $\mathbb{Z}$, *endX* : $\mathbb{Z}$, *endY* : $\mathbb{Z}$)



3.4.2. A backtrackIter algoritmus

A `backtrackIter` visszalépéses keresést alkalmazva generál a labirintushoz egy megoldást, azaz egy összekötő utat a bejárat és a kijárat között. Az algoritmus szintén egy iteratíván megvalósított a mélységi bejárás. Ezen algoritmus közvetlenül a labirintus legenerálása után kerül lefuttatásra.

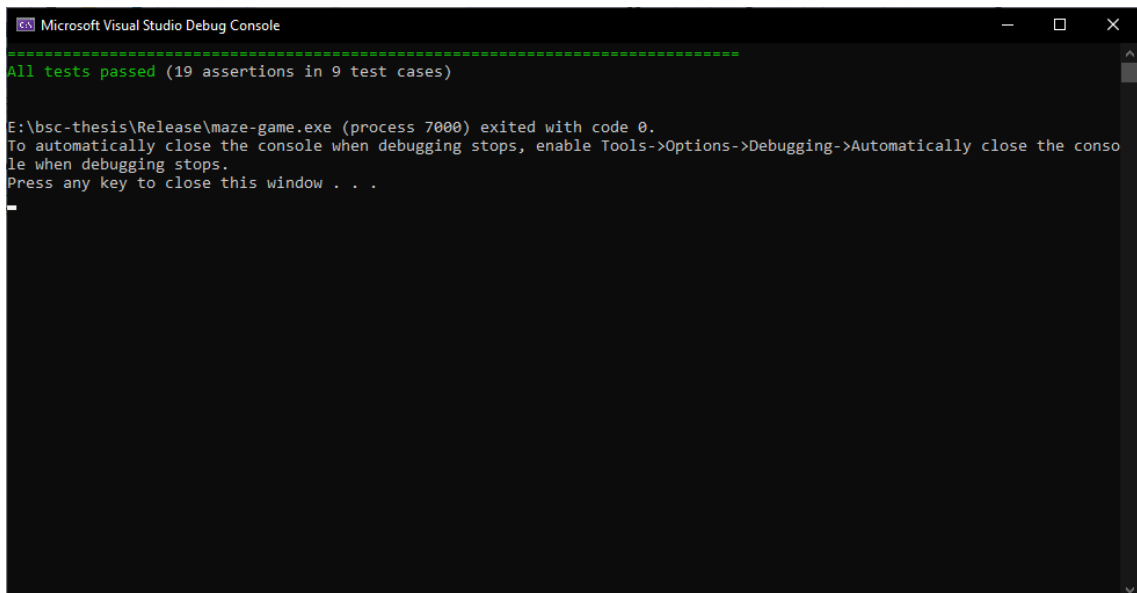
backtrackIter(*startX* : $\mathbb{Z}$, *startY* : $\mathbb{Z}$, *endX* : $\mathbb{Z}$, *endY* : $\mathbb{Z}$)

$i := 0..height - 1$					
<table><tr><td colspan="2">$j := 0..width - 1$</td></tr><tr><td colspan="2">$mazeCells[i][j] \rightarrow unVisit()$</td></tr></table>		$j := 0..width - 1$		$mazeCells[i][j] \rightarrow unVisit()$	
$j := 0..width - 1$					
$mazeCells[i][j] \rightarrow unVisit()$					
$currentCell := mazeCells[startY][startX]$					
$finishCell := mazeCells[endY][endX]$					
$currentCell \rightarrow visit()$					
$finishCell \rightarrow setFinish()$					
$currentCell \neq finishCell$					
$neighbors := getUnvisitedAccessibleNeighbors(currentCell)$					
$\neg neighbors.empty()$					
$index := random(0, neighbors.size() - 1)$	$\emptyset$				
$cell := neighbors[index]$					
$cell \rightarrow visit()$					
$currentCell \rightarrow setFinishPath()$					
$stack.push(currentCell)$					
$currentCell := cell$					
continue					
$currentCell \rightarrow unSetFinishPath()$					
$currentCell := stack.top()$					
$stack.pop()$					

3.5. Tesztelés

A szoftver működésének tesztelését az úgynevezett fehér doboz tesztelési módszerrel valósítom meg. Lényege, hogy a teszteseteket a program belső szerkezetének (forráskódjának) figyelembevételével kell implementálni. Így meg lehet győződni egy-egy főbb metódus helyes működéséről, azaz, hogy különböző bemenetekre helyes visszatérési értéket adnak-e. A cél, hogy a szoftver minél alaposabban le legyen tesztelve, erre jó módszer a kimerítő út tesztelés. Itt a lehető legtöbb fajta tesztesetet kell létrehozni egy adott metódusra, hogy az összes szélsőséges eset lefedésre kerüljön.

A tesztelés helyességéről akkor lehet meggyőződni, ha minden teszteset átmegy. A játékmotor unit tesztelése során minden teszteset sikeresen lefutott, ennek eredményét az alábbi ábra szemlélteti.



3.11. ábra. Sikeres tesztelés

A fehér doboz tesztelés megvalósításához a `main.cpp` fájlban unit teszteseteket implementáltam a Catch2 [9] keretrendszer segítségével. A tesztek futtatásához a programot unit teszt módban kell fordítani és elindítani. Ehhez a fájl elején található `CATCH_CONFIG_MAIN` szimbólumot kell definiálni, amely alapértelmezetten ki van kommentezve. A tesztesetek a függvények megfelelő visszatérési értékét, valamint a procedúrák különféle mellékhatásait tesztelik (például fájlok írása és olvasása). A tesztekhez tartozó bemeneti, valamint kimeneti fájlok a `test` mappában érhetők el. A program által végzett tesztek a következő oldalon található táblázat tartalmazza.

Teszteset neve	Leírás
<i>Initializes and saves configuration correctly</i>	Ellenőrzi, hogy az alapértelmezett konfigurációs fájl létrehozásra kerül a megfelelő adatokkal a <code>Config</code> osztály <code>init()</code> és <code>save()</code> metódusainak meghívásával
<i>Map saves state correctly</i>	Ellenőrzi, hogy egy adott pálya állapota megfelelően mentésre kerül az állás mentésekor a <code>Map</code> osztály <code>saveState()</code> metódusával
<i>Map loads entities correctly</i>	Ellenőrzi, hogy egy adott pályafájl betöltése után megfelelően létrejönnek-e a pálya entitásai a <code>Map</code> osztály <code>loadEntities()</code> metódusával
<i>Two entities collide</i>	Ellenőrzi, hogy két egymásba érő entitás <code>checkCollision()</code> metódusa megfelelő értéket vesz fel
<i>Two entities don't collide</i>	Ellenőrzi, hogy két egymástól távol lévő entitás <code>checkCollision()</code> metódusa megfelelő értéket vesz fel
<i>Camera calculates correct position</i>	Ellenőrzi, hogy a kamera koordinátáit tartalmazó vektor megfelelő értékeket vesz fel a paraméterek (<code>yaw</code> , <code>pitch</code>) beállítása után
<i>Editor saves map correctly</i>	Ellenőrzi, hogy egy pályaszerkesztővel létrehozott és mentett pályafájl a megfelelő adatokkal kerül mentésre
<i>Editor loads map correctly</i>	Ellenőrzi, hogy egy pályaszerkesztővel létrehozott és mentett pályafájl blokkjai megfelelően létrejönnek betöltésekor
<i>Editor selects a block</i>	Ellenőrzi, hogy a pályaszerkesztő megfelelően kijelöli a megadott koordinátán elhelyezkedő blokkot

3.2. táblázat. A tesztesetek leírása

4. fejezet

Összegzés

Összességében elmondható, hogy egy olyan játékszoftvert sikerült fejlesztenem, amely egyaránt szórakoztató és tanulságos.

Felhasználói szempontból a játékelmény változatos és élvezetes, ugyanis a játékos mindig újabb kihívások elé kerül, akár egy, vagy többjátékos módról legyen szó. A látványos háromdimenziós vizuális megjelenítés által a felhasználó tényleg egy igazi labirintusban érezheti magát, még ha nem is élethű a grafika. Emellett szabadságot ad az egyéni kreativitás kifejezéséhez a beépített pályaszerkesztő és a konfigurálható labirintusgenerátor.

A fejlesztő pedig megismerheti egy jól strukturált, szabadon bővíthető játékmotor működését, bővítheti a szakmai ismereteit. Belátást nyer OpenGL és az SDL világába, valamint a többszálú programozás és a hálózati kommunikáció működésébe. Gyakorlatban alkalmazva kaphat betekintést egy labirintusgenerátor algoritmus C++ implementációjába és tetszése szerint módosíthat rajta. Emellett számtalan lehetőséget kap arra is, hogy kifejezze a kreativitását a játékmotor továbbfejlesztésével és más, akár hálózatos játékot fejlesszen, ami nem feltétlenül labirintusjáték.

A szoftvert az egyetemen szerzett tudást és szemléletmódot alkalmazva valósítottam meg, a fejlesztés közben sikerült új ismereteket, valamint tapasztalatot szereznem. Számomra izgalmas feladatokkal szolgált és sikerélményt nyújtott egy-egy főbb rész elkészülése. Úgy gondolom, hogy sikerült bővítenem a játékfejlesztői tudásomat, amit majd később alkalmazni tudok az iparban. Remélem, hogy a szakdolgozatommal másoknak segítséget és motivációt adhatok a játékfejlesztésben.

Irodalomjegyzék

- [1] *Simple DirectMedia Layer*. <https://www.libsdl.org/>. (Elérés dátuma 2021. 04. 15.).
- [2] *JSON for Modern C++*. <https://github.com/nlohmann/json>. (Elérés dátuma 2021. 04. 15.).
- [3] *CppThread*. <https://github.com/berndporr/cppThread>. (Elérés dátuma 2021. 04. 15.).
- [4] *Wavefront .obj file*. https://en.wikipedia.org/wiki/Wavefront_.obj_file. (Elérés dátuma 2021. 04. 15.).
- [5] *Depth-first search*. https://en.wikipedia.org/wiki/Depth-first_search. (Elérés dátuma 2021. 04. 15.).
- [6] *Backtracking*. <https://en.wikipedia.org/wiki/Backtracking>. (Elérés dátuma 2021. 04. 15.).
- [7] *gzip-hpp*. <https://github.com/mapbox/gzip-hpp>. (Elérés dátuma 2021. 04. 15.).
- [8] *base64 (C++)*. <https://github.com/ReneNyffenegger/cpp-base64>. (Elérés dátuma 2021. 04. 15.).
- [9] *Catch2*. <https://github.com/catchorg/Catch2>. (Elérés dátuma 2021. 04. 15.).

Ábrák jegyzéke

2.1. A játékprogram főmenüje	7
2.2. Game Over képernyő egyjátékos módban	10
2.3. Jármű kiválasztása	11
2.4. Közepes nehézségű labirintus egyjátékos módban	12
2.5. Többjátékos módban résztvevő játékosok	13
2.6. Pályaszerkesztő 100×100-as generált labirintussal	15
2.7. A szerverprogram konzolablaka futás közben	17
3.1. A Config osztály osztálydiagramja	27
3.2. Az Entity ősosztály és alosztályainak osztálydiagramja	27
3.3. Egy jármű poligonhálójának szerkesztés közben	37
3.4. Egy kocka lapjainak normálvektorai	40
3.5. A Texture osztály osztálydiagramja	45
3.6. A Window osztály osztálydiagramja	46
3.7. A Button osztály osztálydiagramja	47
3.8. A Menu osztály osztálydiagramja	48
3.9. A kisbetűs betűtípust tartalmazó képfájl	50
3.10. A ServerThread osztály osztálydiagramja	53
3.11. Sikeres tesztelés	58

Táblázatok jegyzéke

2.1. A játékmenü leírása	7
2.2. A játék vezérlése	9
2.3. A labirintusban fellelhető tárgyak	12
2.4. A szerver által küldött üzenetek	14
2.5. A pályaszerkesztő eseményeinek leírása	16
2.6. A szerverprogram által kezelt parancsok	19
3.1. A segédfüggvények és metódusok leírása	49
3.2. A tesztesetek leírása	59

Forráskódjegyzék

3.1. A Block struktúra	36
3.2. A Vertex struktúra	37